

Низкозатратное цифровое предыскажение усилителя мощности

Комплексная фазово-эквивариантная spline-memory модель с direct refinement

Аннотация

Исследуется цифровое предыскажение нелинейного усилителя мощности для комплексного сигнала основной полосы. Рабочая модель должна учитывать амплитудно-фазовое искажение и короткую память, но оставаться пригодной для потокового и fixed-point выполнения.

Предложена причинная структура из трёх комплексных кусочно-линейных сплайн-ветвей. Сначала она получает устойчивую начальную калибровку через ILA, затем **те же коэффициенты** уточняются по локальной чувствительности полного каскада DPD → PA. Формула рабочего тракта при этом не меняется:

$$z[n] = \sum_{m=0}^2 x[n-m] C_m(|x[n-m]|).$$

На BlackBox validation direct refinement улучшил NMSE исходной ILA-модели с **−17.192 до −21.588 dB**. На заранее зарегистрированном untouched test waveform получено **−21.545 dB**, или **+4.337 dB** относительно ILA. Стоимость осталась прежней: **27 вещественных умножений, 28 сложений, 3 вычисления модуля и 6 чтений LUT** на комплексный отсчёт. При 12-bit программной арифметике потеря составила **0.014 dB**.

Граница доказательности: все каскадные результаты получены через зафиксированный программный PA evaluator. Предыскажённый сигнал пока не подавался на физический усилитель. Notebook содержит встроенную копию train/validation capture; перед внешней отправкой необходимо подтвердить право распространения данных.

1. Исследовательский вопрос и вклад работы

Исследовательский вопрос

Можно ли существенно уменьшить ошибку каскада, не добавляя ветви, задержки и нелинейные операции в уже дешёвый spline-memory DPD?

Две проверяемые гипотезы

1. Короткая delay line и комплексные радиальные сплайны дают достаточно выразительный low-cost fast path:

$$z[n] = \sum_{m=0}^2 x[n-m] C_m(|x[n-m]|).$$

2. Основной остаток после ILA связан не только с архитектурой, но и с несовпадением ILA-функции потерь и конечной задачи каскада. Поэтому локальный direct refinement существующих коэффициентов может дать большой выигрыш без роста inference cost.

Практический вклад

1. одна комплексная фазово-эквивариантная модель вместо независимых I/Q;
2. только два активных spline-узла на ветвь и отсчёт;
3. ILA как быстрое устойчивое начальное решение;
4. Якобиан, вычисленный конечными разностями, и регуляризованное direct-обновление;
5. отдельные train-fit и train-advisor блоки, validation без настройки шага;
6. one-shot untouched waveform после фиксации модели и критерия;
7. 16/14/12-bit проверка и точная streaming equivalence;
8. отрицательные структурные расширения сохранены как ablation evidence.

Работа не заявляет доказанную новую архитектуру относительно всей литературы. Результат — контролируемая комбинация известных методов и воспроизводимое доказательство её performance-complexity trade-off на данном evaluator.

Суть идеи простыми словами

Рабочая формула DPD остаётся одной и той же до и после улучшения. Мы не добавляем новые ветви или нейроны. Вместо этого сначала получаем хороший начальный набор коэффициентов, а потом подправляем их так, чтобы уменьшалась ошибка именно на выходе всей цепочки DPD → PA. Поэтому калибровка становится точнее, а стоимость обработки каждого отсчёта не растёт.

```
In [1]: # Базовые импорты и настройки воспроизводимости.
from __future__ import annotations

import base64
import io
import hashlib
import json
import time
from pathlib import Path
```

```

import matplotlib.pyplot as plt
import numpy as np

np.random.seed(0)
np.set_printoptions(precision=5, suppress=True)
plt.style.use("seaborn-v0_8-whitegrid")
print("NumPy:", np.__version__)

# Единое оформление всех исследовательских рисунков.
plt.rcParams.update({
    "figure.dpi": 110,
    "axes.titlesize": 13,
    "axes.labelsize": 11,
    "legend.fontsize": 9,
})

```

NumPy: 2.5.1

```

In [2]: # Физическая постановка: DPD предварительно изменяет desired-сигнал.
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch

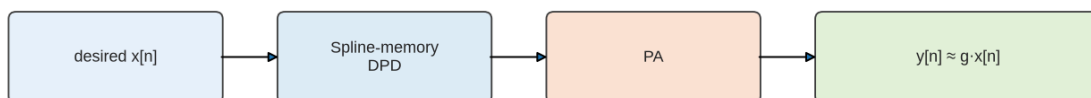
fig, ax = plt.subplots(figsize=(12.5, 3.0))
ax.set_xlim(0, 12.5)
ax.set_ylim(0, 3)
ax.axis("off")

blocks = [
    (0.35, 1.02, 2.15, 0.95, "desired x[n]", "#E8F1FB"),
    (3.15, 1.02, 2.15, 0.95, "Spline-memory\nDPD", "#DDEBF7"),
    (5.95, 1.02, 2.15, 0.95, "PA", "#FCE4D6"),
    (8.75, 1.02, 2.90, 0.95, "y[n] ≈ g·x[n]", "#E2F0D9"),
]

for x0, y0, width, height, label, color in blocks:
    patch = FancyBboxPatch(
        (x0, y0), width, height,
        boxstyle="round,pad=0.04,rounding_size=0.08",
        facecolor=color, edgecolor="0.25", linewidth=1.2,
    )
    ax.add_patch(patch)
    ax.text(x0 + width / 2, y0 + height / 2, label, ha="center", va="center")
for start, end in [(2.5, 1.5), (3.15, 1.5)], ((5.3, 1.5), (5.95, 1.5)), ((8.75, 1.5), (9.4, 1.5))]:
    ax.add_patch(FancyArrowPatch(start, end, arrowstyle="-|>", mutation_scale=20))
ax.text(6.25, 2.45, "Цель: подавить нелинейные искажения после каскада", ha="center", va="bottom")
plt.tight_layout()
plt.show()

```

Цель: подавить нелинейные искажения после каскада



2. Откуда возник выбранный подход

В основе решения лежат четыре направления предыдущих работ.

Источник идеи	Что использовано в данной работе
Memory Polynomial и GMP	Комплексные нелинейные признаки с короткой памятью; GMP используется только как вспомогательная модель PA.
Indirect Learning Architecture	Постдистортер обучается по парам $y/g \rightarrow x$, но финальный DPD проверяется на новом desired-входе.
Spline/LUT DPD	Локальные комплексные control points и интерполяция между соседними узлами дают регулярный малозатратный тракт.
Воспроизводимые DPD-frameworks	Отделение обучения PA evaluator от калибровки и проверки DPD, фиксированные split и метрики.

Особенно близка структурная идея spline memory polynomial из работы P. P. Campro et al.: параллельные комплексные spline-LUT ветви позволяют учесть память при существенно меньшей стоимости, чем полный полиномиальный словарь. В нашей реализации используются линейные сплайны, квантильные узлы и одно комплексное ridge-решение.

Ключевые первичные источники приведены в конце notebook. Их результаты не переносятся в нашу таблицу качества: разные PA и протоколы нельзя смешивать в единый рейтинг.

3. Экспериментальный протокол

Исследование разделено на три этапа.

А. Идентификация вспомогательной модели PA

$$x_{train} \rightarrow \hat{P} \rightarrow \hat{y}_{train} \approx y_{train}.$$

После обучения $\hat{P}$ замораживается. Его стоимость не входит в бюджет рабочего DPD.

В. Начальная ILA-калибровка

$$y_{train}/g \rightarrow D_{post} \rightarrow x_{train}.$$

ILA даёт быстрое линейное по коэффициентам начальное решение. Финальная проверка при этом всегда начинается с desired-сигнала:

$$x \rightarrow DPD \rightarrow \hat{P} \rightarrow \hat{y} \quad \text{vs.} \quad gx.$$

C. Direct refinement

Не меняя сплайн-архитектуру, коэффициенты уточняются по ошибке полного каскада $\hat{P}(D(x)) - gx$. Измеренный y_{val} не подаётся на вход DPD. Validation используется только для итоговой диагностики зафиксированного числа итераций, а не для вычисления обновлений или ранней остановки.

```
In [3]: # Схема потока данных и границы обучения.
fig, ax = plt.subplots(figsize=(13.5, 4.2))
ax.set_xlim(0, 13.5)
ax.set_ylim(0, 4.2)
ax.axis("off")

def draw_box(x0, y0, text, color, width=2.3):
    box = FancyBboxPatch((x0, y0), width, 0.72, boxstyle="round,pad=0.04",
                        facecolor=color, edgecolor="0.3")
    ax.add_patch(box)
    ax.text(x0 + width/2, y0 + 0.36, text, ha="center", va="center", fontsize=12)

def arrow(x1, y1, x2, y2):
    ax.add_patch(FancyArrowPatch((x1, y1), (x2, y2), arrowstyle="-|>",
                            mutation_scale=13, color="0.25"))

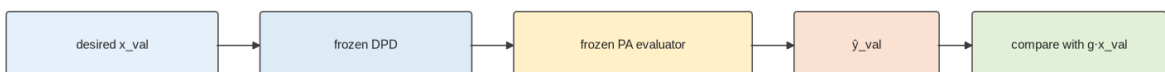
ax.text(0.25, 3.65, "Этап A — train", weight="bold", fontsize=12)
draw_box(0.3, 2.65, "measured x_train", "#E8F1FB")
draw_box(3.15, 2.65, "fit PA evaluator", "#FFF2CC")
draw_box(6.0, 2.65, "measured y_train", "#FCE4D6")
arrow(2.6, 3.01, 3.15, 3.01); arrow(5.45, 3.01, 6.0, 3.01)
ax.text(9.15, 3.01, "после fit: PA evaluator frozen", fontsize=10,
      bbox=dict(boxstyle="round", fc="#E2F0D9", ec="0.4"))

ax.text(0.25, 1.75, "Этап B — validation", weight="bold", fontsize=12)
draw_box(0.3, 0.72, "desired x_val", "#E8F1FB")
draw_box(3.15, 0.72, "frozen DPD", "#DDEBF7")
draw_box(6.0, 0.72, "frozen PA evaluator", "#FFF2CC", width=2.6)
draw_box(9.15, 0.72, "ŷ_val", "#FCE4D6", width=1.55)
draw_box(11.15, 0.72, "compare with g·x_val", "#E2F0D9", width=2.1)
arrow(2.6, 1.08, 3.15, 1.08); arrow(5.45, 1.08, 6.0, 1.08)
arrow(8.6, 1.08, 9.15, 1.08); arrow(10.7, 1.08, 11.15, 1.08)
plt.tight_layout()
plt.show()
```

Этап A — train



Этап B — validation



```
In [4]: # Встроенные train/validation массивы BlackBoxData (сжатый NPZ, base64).
# Генератор notebook подставляет payload перед сохранением файла.
EMBEDDED_DATA_B64 = """UESDBC0AAAAIAAAAIQDrCdnL/////////8PABQAdHJhaw5feF9yY

def load_embedded_splits() -> dict[str, np.ndarray]:
    """Decode the portable train/validation payload from this notebook.

    Returns:
        A dictionary with raw complex ``train_x``, ``train_y``,
        ``val_x`` and ``val_y`` arrays plus a human-readable source.
    """
    binary = base64.b64decode(EMBEDDED_DATA_B64.encode("ascii"))
    with np.load(io.BytesIO(binary), allow_pickle=False) as payload:
        result = {name: np.asarray(payload[name], dtype=np.complex128) for name in
            "train_x_raw", "train_y_raw", "val_x_raw", "val_y_raw"
        }
    result["source"] = "embedded BlackBoxData train/validation payload"
    return result

print("embedded payload ready; source data are portable with this notebook")
```

embedded payload ready; source data are portable with this notebook

4. Данные и дисциплина разбиения

Используется один BlackBox capture с комплексными массивами `x` и `y`:

- train: 92 000 отсчётов, исходный диапазон 5000:97000 ;
- validation: 23 000 отсчётов, 97000:120000 ;
- untouched test: 125 760 отсчётов, начиная с 120000 .

Топология и K исторически выбирались по validation, поэтому validation не является независимой финальной оценкой. После фиксации direct-кандидата, метрики, warm-up и acceptance gate был один раз открыт только желаемый x_{test} . Измеренный y_{test} не открывался: test остаётся проверкой нового waveform через тот же frozen PA evaluator, а не физическим DPD-измерением.

Train/validation встроены для воспроизводимости notebook. Test samples не встраиваются; ниже используется только сохранённый агрегированный one-shot отчёт.

```
In [5]: # Фиксированные параметры запуска. В portable-режиме менять пути не нужно:
# данные уже находятся внутри notebook.
PROJECT_CANDIDATES = [Path.cwd(), *Path.cwd().parents]
PROJECT_ROOT = next(
    (p for p in PROJECT_CANDIDATES
     if (p / "data/private/blackbox_v3/selection").is_dir()),
    Path.cwd(),
)
```

```

SELECTION_DIR = PROJECT_ROOT / "data/private/blackbox_v3/selection"
USE_EMBEDDED_DATA = True
MAT_PATH = None # пример: Path("/content/BlackBoxData.mat")
print("external project data directory detected =", SELECTION_DIR.is_dir())
print("USE_EMBEDDED_DATA =", USE_EMBEDDED_DATA)

```

```

external project data directory detected = True
USE_EMBEDDED_DATA = True

```

```

In [6]: def read_iq_csv(path: Path) -> np.ndarray:
        """Read a two-column ``I,Q`` CSV and return ``I + jQ`` samples.

        Args:
            path: CSV file with a header and two numeric columns.

        Returns:
            One-dimensional finite complex128 vector.
        """
        raw = np.loadtxt(path, delimiter=",", skiprows=1)
        if raw.ndim != 2 or raw.shape[1] < 2:
            raise ValueError(f"Expected I,Q columns in {path}")
        signal = np.asarray(raw[:, 0] + 1j * raw[:, 1], dtype=np.complex128)
        if not np.all(np.isfinite(signal)):
            raise ValueError(f"Non-finite samples in {path}")
        return signal

def load_mat_splits(path: Path) -> dict[str, np.ndarray]:
    """Load the optional MAT fallback and apply the frozen split.

    The source variables are validated, while ``eRef`` is retained only
    for the shape check and is intentionally not used by the model.
    """
    from scipy.io import loadmat

    payload = loadmat(path)
    vectors = {}
    for name in ("x", "y", "eRef"):
        if name not in payload:
            raise ValueError(f"MAT file lacks {name}")
        value = np.asarray(payload[name]).reshape(-1)
        if not np.iscomplexobj(value):
            raise ValueError(f"MAT variable {name} is not complex")
        vectors[name] = np.asarray(value, dtype=np.complex128)
    if len({v.size for v in vectors.values()}) != 1:
        raise ValueError("x, y and eRef must have equal lengths")
    return {
        "train_x_raw": vectors["x"][5000:97000],
        "train_y_raw": vectors["y"][5000:97000],
        "val_x_raw": vectors["x"][97000:120000],
        "val_y_raw": vectors["y"][97000:120000],
        "source": path,
    }

def load_blackbox_splits(selection_dir: Path, mat_path: Path | None) -> dict

```

```

"""Load embedded data or explicit CSV/MAT files with clear errors."""
if USE_EMBEDDED_DATA:
    return load_embedded_splits()
files = {
    "train_x_raw": selection_dir / "train_input.csv",
    "train_y_raw": selection_dir / "train_output.csv",
    "val_x_raw": selection_dir / "val_input.csv",
    "val_y_raw": selection_dir / "val_output.csv",
}
if all(path.is_file() for path in files.values()):
    result = {name: read_iq_csv(path) for name, path in files.items()}
    result["source"] = selection_dir
    return result
if mat_path is not None and Path(mat_path).is_file():
    return load_mat_splits(mat_path)
raise FileNotFoundError(
    "USE_EMBEDDED_DATA=False, но данные не найдены. "
    "Укажите каталог с четырьмя I,Q CSV в SELECTION_DIR или "
    "существующий BlackBoxData.mat в MAT_PATH."
)

raw = load_blackbox_splits(SELECTION_DIR, MAT_PATH)
train_x_raw, train_y_raw = raw["train_x_raw"], raw["train_y_raw"]
val_x_raw, val_y_raw = raw["val_x_raw"], raw["val_y_raw"]
assert train_x_raw.size == train_y_raw.size == 92000
assert val_x_raw.size == val_y_raw.size == 23000
print("source:", raw["source"])
print("train:", train_x_raw.shape, "validation:", val_x_raw.shape)
print("raw train input peak:", np.max(np.abs(train_x_raw)))

```

source: embedded BlackBoxData train/validation payload
train: (92000,) validation: (23000,)
raw train input peak: 18336.09364068585

5. Выравнивание, нормировка и комплексный gain

Все параметры preprocessing оцениваются **только по train**:

1. integer delay максимизирует модуль нормированной комплексной корреляции;
2. общий масштаб равен максимуму $|x_{train}|$;
3. комплексный линейный gain оценивается как

$$g = \frac{\sum_n x^*[n]y[n]}{\sum_n |x[n]|^2}.$$

Те же delay, scale и gain затем без переоценки применяются к validation. Fractional delay ниже показан только как диагностика параболической интерполяцией вокруг корреляционного максимума.


```

In [7]: # Alignment, normalization и gain оцениваются только на train.
def overlap_at_delay(x: np.ndarray, y: np.ndarray, delay: int):
    """Return overlapping vectors when y is delayed by ``delay`` vs x."""
    if delay > 0:
        return x[:-delay], y[delay:]
    if delay < 0:
        return x[-delay:], y[:delay]
    return x, y

def normalized_alignment_score(x: np.ndarray, y: np.ndarray, delay: int) ->
    """Return magnitude of normalized complex correlation at one delay."""
    xa, ya = overlap_at_delay(x, y, delay)
    denominator = np.sqrt(np.vdot(xa, xa).real * np.vdot(ya, ya).real)
    return float(abs(np.vdot(xa, ya)) / denominator)

ALIGNMENT_DELAYS = np.arange(-32, 33)
alignment_scores = np.array([
    normalized_alignment_score(train_x_raw, train_y_raw, int(delay))
    for delay in ALIGNMENT_DELAYS
])
integer_delay = int(ALIGNMENT_DELAYS[np.argmax(alignment_scores)])
peak_index = int(np.argmax(alignment_scores))
fractional_offset = 0.0
if 0 < peak_index < alignment_scores.size - 1:
    left_score, peak_score, right_score = alignment_scores[peak_index - 1:peak_index + 2]
    curvature = left_score - 2.0 * peak_score + right_score
    if abs(curvature) > 1e-15:
        fractional_offset = float(0.5 * (left_score - right_score) / curvature)
fractional_delay_diagnostic = integer_delay + fractional_offset

# Один train-only integer delay применяется без переоценки на validation.
train_x_aligned, train_y_aligned = overlap_at_delay(train_x_raw, train_y_raw, integer_delay)
val_x_aligned, val_y_aligned = overlap_at_delay(val_x_raw, val_y_raw, integer_delay)

TRAIN_INPUT_PEAK = float(np.max(np.abs(train_x_aligned)))
train_x = train_x_aligned / TRAIN_INPUT_PEAK
train_y = train_y_aligned / TRAIN_INPUT_PEAK
val_x = val_x_aligned / TRAIN_INPUT_PEAK
val_y = val_y_aligned / TRAIN_INPUT_PEAK

gain = np.vdot(train_x, train_y) / np.vdot(train_x, train_x)
print(f"train-only integer delay = {integer_delay} samples")
print(f"fractional-delay diagnostic = {fractional_delay_diagnostic:+.6f} samples")
print(f"alignment peak score = {alignment_scores[peak_index]:.6f}")
print(f"train-only scale = {TRAIN_INPUT_PEAK:.6f}")
print(f"train-only complex gain = {gain.real:.8f} {gain.imag:+.8f}j")
print("normalized peaks:", * (f"{np.max(np.abs(v)):.6f}" for v in (train_x, train_y)))

fig, ax = plt.subplots(figsize=(9.5, 3.6))
ax.plot(ALIGNMENT_DELAYS, alignment_scores, marker=".", ms=4)
ax.axvline(integer_delay, color="tab:red", ls="--", label=f"выбрано: {integer_delay}")
ax.set(xlabel="задержка y относительно x [отсчёты]", ylabel="|нормированная корреляция|")
ax.title("Диагностика временного выравнивания только по train")

```

```
ax.legend()
plt.tight_layout()
plt.show()
```

train-only integer delay = 0 samples
fractional-delay diagnostic = -0.014700 samples (not applied)
alignment peak score = 0.986855
train-only scale = 18336.093641
train-only complex gain = 0.99962078 +0.00067656j
normalized peaks: 1.000000 1.093883 0.971956 1.085530



```
In [8]: # Проверка сопоставимости train/validation и объяснение AM/AM-разброса.
def quantile_binned_statistics(radius, values, bin_count=32):
    """Return conditional median and 10/90 percentiles in quantile bins."""
    radius = np.asarray(radius, dtype=float)
    values = np.asarray(values, dtype=float)
    if radius.shape != values.shape or radius.ndim != 1:
        raise ValueError("radius and values must be equal one-dimensional arrays")
    edges = np.unique(np.quantile(radius, np.linspace(0.0, 1.0, bin_count + 1)))
    centers, medians, lower, upper, counts = [], [], [], [], []
    for index, (left, right) in enumerate(zip(edges[:-1], edges[1:])):
        mask = (radius >= left) & (radius < right)
        if index == edges.size - 2:
            mask = (radius >= left) & (radius <= right)
        if np.count_nonzero(mask) < 10:
            continue
        centers.append(np.median(radius[mask]))
        medians.append(np.median(values[mask]))
        lower.append(np.quantile(values[mask], 0.10))
        upper.append(np.quantile(values[mask], 0.90))
        counts.append(np.count_nonzero(mask))
    return {name: np.asarray(value) for name, value in {
        "center": centers, "median": medians, "q10": lower,
        "q90": upper, "count": counts,
    }.items()}

sample = slice(None, None, 25)
fig, axes = plt.subplots(1, 3, figsize=(16.5, 4.6))

amplitude_bins = np.linspace(
    0, max(np.max(np.abs(train_x)), np.max(np.abs(val_x))), 70
)
```

```

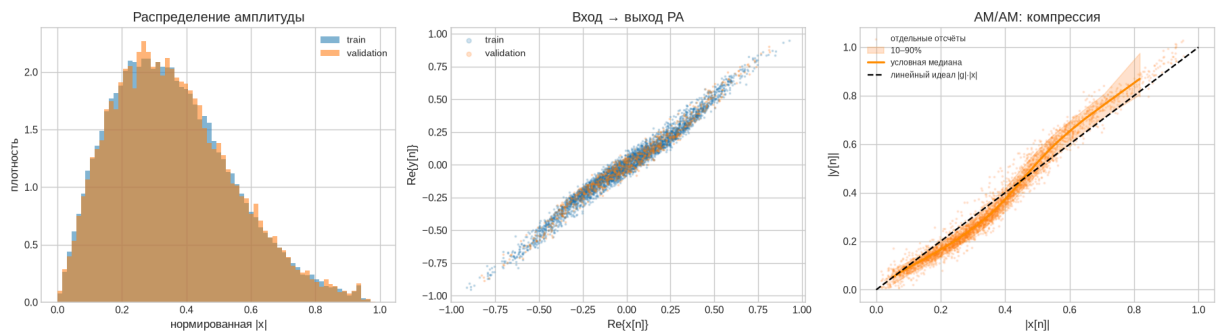
axes[0].hist(np.abs(train_x), bins=amplitude_bins, density=True,
             alpha=0.55, label="train")
axes[0].hist(np.abs(val_x), bins=amplitude_bins, density=True,
             alpha=0.55, label="validation")
axes[0].set(xlabel="нормированная |x|", ylabel="плотность",
            title="Распределение амплитуды")
axes[0].legend()

# Возвращён исходный scatter: один маркер соответствует одному временному от
axes[1].scatter(train_x.real[sample], train_y.real[sample], s=2,
               alpha=0.20, color="tab:blue", label="train")
axes[1].scatter(val_x.real[sample], val_y.real[sample], s=2,
               alpha=0.20, color="tab:orange", label="validation")
axes[1].set(xlabel="Re{x[n]}", ylabel="Re{y[n]}",
            title="Вход → выход РА")
axes[1].legend(markerscale=3)

# Исходные точки AM/AM оставлены оранжевыми; диапазон 10–90% не скрывает их.
train_amam = quantile_binned_statistics(np.abs(train_x), np.abs(train_y))
axes[2].scatter(np.abs(train_x)[sample], np.abs(train_y)[sample], s=2,
               alpha=0.13, color="tab:orange", label="отдельные отсчёты")
axes[2].fill_between(train_amam["center"], train_amam["q10"],
                    train_amam["q90"], color="tab:orange", alpha=0.20,
                    label="10–90%")
axes[2].plot(train_amam["center"], train_amam["median"],
             color="darkorange", lw=2.0, label="условная медиана")
r_grid = np.linspace(0, max(np.max(np.abs(train_x)), np.max(np.abs(val_x))),
                    axes[2].plot(r_grid, abs(gain) * r_grid, color="black", ls="--",
                                label="линейный идеал |g|·|x|")
axes[2].set(xlabel="|x[n]|", ylabel="|y[n]|", title="AM/AM: компрессия")
axes[2].legend(fontsize=8)
plt.tight_layout()
plt.show()

print("Минимум отсчётов в AM/AM bin:", int(train_amam["count"].min()))
print("Полоса 10–90% показывает наблюдаемый разброс, а не доверительный интервал")

```



Минимум отсчётов в AM/AM bin: 2875

Полоса 10–90% показывает наблюдаемый разброс, а не доверительный интервал.

Как читать точки на графиках РА

- На графике «Вход → выход РА» каждая точка соответствует одному и тому же временному отсчёту n : по горизонтали показана $\text{Re}\{x[n]\}$, по вертикали —

$\text{Re}\{y[n]\}$. Синие точки относятся к train, оранжевые — к validation. Это проекция комплексных I/Q-отсчётов, а не отдельные символы созвездия.

- На графике **AM/AM** каждая оранжевая точка — пара $(|x[n]|, |y[n]|)$. Поэтому вертикальный разброс при одном $|x|$ может появляться из-за памяти РА и различного соседнего I/Q-контекста.
- Оранжевая полоса 10–90% содержит центральные 80% выходных амплитуд внутри каждого диапазона $|x|$. Она делает разброс читаемым, но не заменяет и не «исправляет» исходные точки.

Как читать разброс

График $\text{Re}\{g \cdot x\} \rightarrow \text{Re}\{y\}$ и AM/AM являются проекциями системы с памятью на две координаты. Реальный РА описывается скорее как

$$y[n] = f(x[n], x[n-1], x[n-2], \dots),$$

поэтому одинаковое текущее $|x[n]|$ не обязано давать одинаковое $|y[n]|$. Условная медиана показывает центральную AM/AM-зависимость, а полоса 10–90% сохраняет информацию о памяти, фазовом окружении и измерительном разбросе. Такое агрегирование делает рисунок читаемее, но не выдаётся за физическое уменьшение дисперсии.

6. Метрика качества

Основная временная метрика — pooled complex NMSE:

$$\text{NMSE}_{dB} = 10 \log_{10} \frac{\sum_n |\hat{y}[n] - y_{ref}[n]|^2}{\sum_n |y_{ref}[n]|^2}.$$

Более отрицательное значение означает меньшую нормированную мощность ошибки. Первые causal warm-up отсчёты исключаются. Отдельная QAM EVM не вычисляется: без символов, демодуляции, синхронизации и эквализации это было бы неверной трактовкой нормированной временной ошибки.

Интуитивная трактовка

NMSE сравнивает мощность ошибки с мощностью желаемого сигнала. Например, -20 dB соответствует относительной мощности ошибки около 1%. Поэтому -21.6 dB лучше, чем -17.2 dB: в первом случае ошибка занимает меньшую долю мощности сигнала.

```
In [9]: def pooled_nmse_db(estimate: np.ndarray, reference: np.ndarray, warmup: int
        """Return pooled complex NMSE in dB after a causal warm-up.
```

```

Args:
    estimate: Predicted complex output.
    reference: Desired/reference complex output.
    warmup: Number of record-start samples excluded from scoring.

Returns:
    ``10*log10(MSE / reference_power)``.
"""
estimate = np.asarray(estimate, dtype=np.complex128)
reference = np.asarray(reference, dtype=np.complex128)
if estimate.shape != reference.shape or estimate.ndim != 1:
    raise ValueError("metric vectors must be equal 1-D arrays")
e = estimate[warmup:] - reference[warmup:]
ref = reference[warmup:]
return float(10.0 * np.log10(np.mean(np.abs(e) ** 2) / np.mean(np.abs(re

def normalized_rms_error_db(estimate: np.ndarray, reference: np.ndarray, war
    """Return time-domain normalized RMS error; equal to NMSE in dB."""
    e = np.asarray(estimate)[warmup:] - np.asarray(reference)[warmup:]
    ref = np.asarray(reference)[warmup:]
    return float(20.0 * np.log10(np.sqrt(np.mean(np.abs(e) ** 2)) / np.sqrt(

def summary(signal: np.ndarray) -> dict[str, float]:
    """Summarize RMS amplitude, peak amplitude and PAPR of a signal."""
    power = float(np.mean(np.abs(signal) ** 2))
    peak = float(np.max(np.abs(signal)))
    return {
        "rms": float(np.sqrt(power)),
        "peak": peak,
        "papr_db": float(10.0 * np.log10(peak * peak / power)),
    }

def metric_row(name: str, estimate: np.ndarray, reference: np.ndarray, warmu
    """Build one readable report row from the common metric functions."""
    return {
        "name": name,
        "nmse_db": pooled_nmse_db(estimate, reference, warmup),
        **summary(estimate),
    }

# Algebraic identity check; this is why no independent EVM column is shown.
_metric_probe = np.array([1 + 1j, 0.5 - 0.2j])
assert abs(pooled_nmse_db(0.9 * _metric_probe, _metric_probe)
           - normalized_rms_error_db(0.9 * _metric_probe, _metric_probe)) <

```

7. Вспомогательная модель усилителя

Для воспроизводимого каскада используется причинный Generalized Memory Polynomial:

$$\begin{aligned}\hat{y}[n] = & \sum_{k,q} a_{kq} x[n-q] |x[n-q]|^k \\ & + \sum_{k,q,l} b_{kql} x[n-q] |x[n-q-l]|^k \\ & + \sum_{k,q,l} c_{kql} x[n-q] |x[n-q+l]|^k.\end{aligned}$$

Leading-envelope terms допускаются только когда итоговая зависимость остаётся причинной. Коэффициенты оцениваются комплексной ridge-регрессией на train и после fit замораживаются. Это **evaluator эксперимента**, а не часть рабочего DPD; его стоимость не входит в fast path.

Зачем вообще нужна модель PA

В этом notebook нельзя многократно подавать каждый пробный сигнал на физический усилитель. Поэтому по измеренным парам $x \rightarrow y$ строится его программная копия. Она позволяет быстро сравнивать варианты DPD. GMP в формуле ниже — это набор нелинейных членов с разными задержками; он описывает как мгновенную нелинейность, так и короткую память PA.

```
In [10]: def causal_delay(signal: np.ndarray, delay: int) -> np.ndarray:
    """Apply a causal integer delay with zero padding at record start."""
    out = np.zeros_like(signal, dtype=np.complex128)
    if delay == 0:
        out[:] = signal
    elif delay > 0:
        out[delay:] = signal[:-delay]
    return out

def gmp_terms(ka=9, la=9, kb=3, lb=7, mb=3, kc=3, lc=7, mc=3):
    """Enumerate causal GMP terms as ``(branch, order, q, envelope_q)``."""
    terms = []
    for exponent in range(ka):
        for q in range(la):
            terms.append(("aligned", exponent, q, q))
    for exponent in range(1, kb + 1):
        for q in range(lb):
            for lag in range(1, mb + 1):
                terms.append(("lagging", exponent, q, q + lag))
    for exponent in range(1, kc + 1):
        for q in range(lc):
            for lead in range(1, mc + 1):
                if lead <= q: # causal-leading policy
                    terms.append(("leading", exponent, q, q - lead))
    return tuple(terms)

def gmp_design(signal: np.ndarray, terms: tuple) -> np.ndarray:
    """Construct the dense complex GMP design matrix in term order."""
```

```

signal = np.asarray(signal, dtype=np.complex128)
columns = []
for _branch, exponent, signal_delay, envelope_delay in terms:
    delayed = causal_delay(signal, signal_delay)
    if exponent == 0:
        columns.append(delayed)
    else:
        envelope = np.abs(causal_delay(signal, envelope_delay))
        columns.append(delayed * envelope ** exponent)
return np.column_stack(columns)

def fit_complex_ridge(design: np.ndarray, target: np.ndarray, ridge: float)
    """Fit complex coefficients with column scaling and ridge regularization

    The data term is normalized by the number of samples, matching the
    frozen PA-selection protocol used for the BlackBox experiment.
    """
    design = np.asarray(design, dtype=np.complex128)
    target = np.asarray(target, dtype=np.complex128)
    column_rms = np.sqrt(np.mean(np.abs(design) ** 2, axis=0))
    if np.any(column_rms <= 0) or not np.all(np.isfinite(column_rms)):
        raise ValueError("invalid design column scale")
    # The project protocol minimizes mean squared error + ridge:
    #  $\|Dc - q\|^2/N + \text{ridge} * \|c\|^2$ . Dividing both data and target
    # by  $\sqrt{N}$  is therefore part of the definition, not cosmetic
    # numerical scaling.
    normalization = np.sqrt(float(design.shape[0]))
    scaled = design / column_rms / normalization
    normalized_target = target / normalization
    if ridge > 0:
        regularizer = np.sqrt(ridge) * np.eye(
            scaled.shape[1], dtype=np.complex128
        )
        augmented = np.vstack((scaled, regularizer))
        augmented_target = np.concatenate(
            (normalized_target, np.zeros(scaled.shape[1], dtype=np.complex128)
        )
    else:
        augmented, augmented_target = scaled, normalized_target
    coeff_scaled = np.linalg.lstsq(augmented, augmented_target, rcond=None)[0]
    return coeff_scaled / column_rms

# Frozen evaluator configuration selected on the BlackBox validation split.
# The PA is auxiliary: its complexity is not the deployment DPD budget.
pa_terms = gmp_terms()
assert len(pa_terms) == 189
# Forward identification: measured PA input x -> measured output y.
started = time.perf_counter()
pa_design = gmp_design(train_x, pa_terms)
pa_coefficients = fit_complex_ridge(pa_design, train_y, ridge=1e-10)
pa_fit_seconds = time.perf_counter() - started
del pa_design
pa_train_hat = gmp_design(train_x, pa_terms) @ pa_coefficients
pa_val_design = gmp_design(val_x, pa_terms)

```

```

pa_val_hat = pa_val_design @ pa_coefficients
del pa_val_design
print(f"GMP terms: {len(pa_terms)} complex coefficients")
print(f"fit time: {pa_fit_seconds:.2f} s")
print(f"PA validation fidelity: {pooled_nmse_db(pa_val_hat, val_y, warmup=11

```

GMP terms: 189 complex coefficients
fit time: 8.12 s
PA validation fidelity: -28.590 dB NMSE

```

In [11]: # До DPD: каскад frozen PA на желаемом validation input.
PA_WARMUP = 9
DPD_WARMUP = 2
CASCADE_WARMUP = PA_WARMUP + DPD_WARMUP
ideal_val = gain * val_x
measured_no_dpd = metric_row("measured y (diagnostic)", val_y, ideal_val, CA
surrogate_no_dpd = metric_row("frozen PA, no DPD", pa_val_hat, ideal_val, CA
print(measured_no_dpd)
print(surrogate_no_dpd)

```

```

{'name': 'measured y (diagnostic)', 'nmse_db': -15.78064467142287, 'rms': 0.
4096629648882422, 'peak': 1.0855304876366918, 'papr_db': 8.464306429911977}
{'name': 'frozen PA, no DPD', 'nmse_db': -16.010350840472285, 'rms': 0.40948
11906023182, 'peak': 1.0540003577118497, 'papr_db': 8.212136018077079}

```

8. Предлагаемая spline-memory DPD

8.1 Архитектура

Для ветвей $m \in \{0, 1, 2\}$:

$$z[n] = \sum_{m=0}^2 x[n-m] C_m(r_m[n]), \quad r_m[n] = |x[n-m]|.$$

$C_m(r)$ — комплексная кусочно-линейная функция. Для $r \in [r_k, r_{k+1}]$:

$$C_m(r) = (1 - \alpha) C_{m,k} + \alpha C_{m,k+1}, \quad \alpha = \frac{r - r_k}{r_{k+1} - r_k}.$$

Таким образом, один комплексный коэффициент совместно задаёт амплитудную и фазовую коррекцию, а на отсчёте активны только два соседних узла каждой ветви.

8.2 Фазовая эквивариантность

Пусть $x'[n] = x[n]e^{j\varphi}$. Тогда $|x'[n-m]| = |x[n-m]|$, следовательно:

$$\begin{aligned}
D(xe^{j\varphi})[n] &= \sum_m x[n-m] e^{j\varphi} C_m(|x[n-m]|) \\
&= e^{j\varphi} D(x)[n].
\end{aligned}$$

Модель не привязана к произвольной ориентации I/Q-плоскости.

8.3 Почему квантильные узлы

Квантильное размещение даёт каждому интервалу примерно одинаковое число калибровочных отсчётов. Это улучшает обусловленность по сравнению с равномерными узлами, когда большая часть редких высокоамплитудных интервалов получает мало данных. Цена — менее равномерное разрешение по амплитуде, поэтому число узлов всё равно выбирается экспериментально.

Интуитивное объяснение

Модель состоит из трёх одинаково устроенных ветвей. Первая работает с текущим отсчётом, вторая — с предыдущим, третья — с отсчётом два шага назад. Так учитывается короткая память усилителя.

Внутри каждой ветви стоит сплайн — ломаная линия, заданная небольшим числом узлов. Для каждого отсчёта нужно прочитать только два соседних узла и провести между ними линейную интерполяцию. Именно поэтому 24 узла не означают 24 умножения на каждую ветвь.

```
In [12]: def make_quantile_knots(signal: np.ndarray, count: int) -> np.ndarray:
    """Place strictly increasing interpolation knots by amplitude quantiles.
    radius = np.abs(np.asarray(signal, dtype=np.complex128))
    unit = np.linspace(0.0, 1.0, count)
    knots = np.quantile(radius, unit)
    knots[0] = 0.0
    knots[-1] = np.max(radius)
    knots = np.unique(knots)
    if knots.size < 2 or np.any(np.diff(knots) <= 0):
        raise ValueError("quantile knots are not strictly increasing")
    return knots

def spline_coordinates(radius: np.ndarray, knots: np.ndarray):
    """Return left-knot indices and local linear interpolation weights."""
    clipped = np.clip(np.asarray(radius, dtype=float), knots[0], knots[-1])
    left = np.searchsorted(knots, clipped, side="right") - 1
    left = np.clip(left, 0, knots.size - 2)
    weight = (clipped - knots[left]) / (knots[left + 1] - knots[left])
    return left.astype(np.int64), weight

def spline_memory_design(signal: np.ndarray, knots: np.ndarray, branches: tuple) -> np.ndarray:
    """Build  $\Phi[n, BK+k] = x[n-m] B_k(|x[n-d]|)$  for spline-memory DPD."""
    signal = np.asarray(signal, dtype=np.complex128)
    columns = []
    for signal_delay, envelope_delay in branches:
        delayed_signal = causal_delay(signal, signal_delay)
        delayed_envelope = causal_delay(np.abs(signal), envelope_delay)
        left, weight = spline_coordinates(np.abs(delayed_envelope), knots)
        basis = np.zeros((signal.size, knots.size), dtype=float)
        rows = np.arange(signal.size)
```

```

        basis[rows, left] = 1.0 - weight
        basis[rows, left + 1] += weight
        columns.extend((delayed_signal[:, None] * basis).T)
    return np.column_stack(columns)

def fit_spline_memory_dpd(calibration_input, target, knots, branches, ridge):
    """Fit the ILA spline-memory coefficients on the causal steady region."""
    warmup = max(max(b) for b in branches)
    design = spline_memory_design(calibration_input, knots, branches)[warmup:]
    # The frozen project protocol intentionally uses the raw spline
    # dictionary here (no column scaling), with the same normalized
    # objective and ridge as factorize_spline_group().
    normalization = np.sqrt(float(design.shape[0]))
    solve_design = design / normalization
    solve_target = target[warmup:] / normalization
    augmented = np.vstack(
        (solve_design, np.sqrt(ridge) * np.eye(design.shape[1], dtype=np.complex128))
    )
    augmented_target = np.concatenate(
        (solve_target, np.zeros(design.shape[1], dtype=np.complex128))
    )
    flat = np.linalg.lstsq(augmented, augmented_target, rcond=None)[0]
    return flat.reshape(len(branches), knots.size), warmup

def predict_spline_memory(signal, knots, branches, coefficients):
    """Evaluate the causal spline-memory DPD for one independent record."""
    signal = np.asarray(signal, dtype=np.complex128)
    output = np.zeros(signal.size, dtype=np.complex128)
    for branch_index, (signal_delay, envelope_delay) in enumerate(branches):
        delayed_signal = causal_delay(signal, signal_delay)
        delayed_envelope = causal_delay(signal, envelope_delay)
        left, weight = spline_coordinates(np.abs(delayed_envelope), knots)
        c = coefficients[branch_index]
        correction = c[left] + weight * (c[left + 1] - c[left])
        output += delayed_signal * correction
    return output

```

```

In [13]: # Иллюстрация локальной поддержки линейного spline basis.
illustration_knots = np.array([0.0, 0.10, 0.23, 0.40, 0.62, 1.0])
illustration_r = np.linspace(0, 1, 700)
left_i, weight_i = spline_coordinates(illustration_r, illustration_knots)
basis_i = np.zeros((illustration_r.size, illustration_knots.size))
rows_i = np.arange(illustration_r.size)
basis_i[rows_i, left_i] = 1 - weight_i
basis_i[rows_i, left_i + 1] += weight_i

fig, axes = plt.subplots(1, 2, figsize=(13.5, 4.2))
for index in range(illustration_knots.size):
    axes[0].plot(illustration_r, basis_i[:, index], label=f"B{index}")
axes[0].set(xlabel="амплитуда r", ylabel="B_k(r)",
            title="Локальные линейные базисные функции")
axes[0].legend(ncols=3)

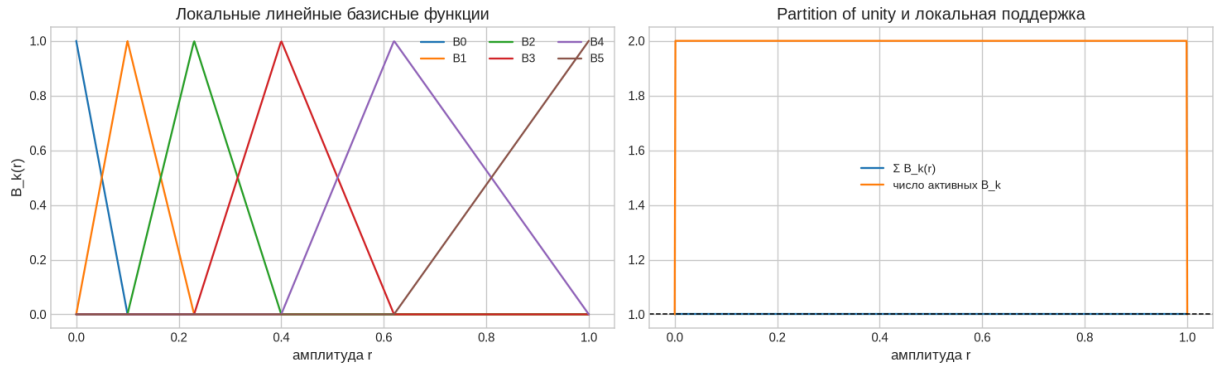
active_count = np.sum(basis_i > 1e-14, axis=1)

```

```

axes[1].plot(illustration_r, basis_i.sum(axis=1), label="Σ B_k(r)")
axes[1].plot(illustration_r, active_count, label="число активных B_k")
axes[1].axhline(1, color="black", ls="--", lw=1)
axes[1].set(xlabel="амплитуда r", title="Partition of unity и локальная подд
axes[1].legend()
assert np.max(np.abs(basis_i.sum(axis=1) - 1.0)) < 1e-14
assert np.max(active_count) <= 2
plt.tight_layout()
plt.show()

```



9. Калибровка коэффициентов

Используется Indirect Learning Architecture только как метод калибровки. На train формируется нормированный выход РА

$$u[n] = \frac{y[n]}{g},$$

и локальные признаки

$$\Phi_{n,(m,k)} = u[n - m] B_k(|u[n - m]|).$$

Коэффициенты находятся из выпуклой задачи

$$\hat{\mathbf{c}} = \arg \min_{\mathbf{c}} \frac{1}{N} \|\Phi \mathbf{c} - \mathbf{x}\|_2^2 + \lambda \|\mathbf{c}\|_2^2.$$

При фиксированных узлах это одна комплексная linear least-squares задача. После fit коэффициенты переносятся в predistorter, а validation начинается с desired x_{val} .

Правило выбора модели

Проверяются две причинные топологии и $K \in \{4, 8, 12, 16, 24, 32\}$. Выбирается модель с минимальным числом вещественных коэффициентов среди вариантов, находящихся не далее 0.01 dB от лучшего validation NMSE. Это заранее заданный компромисс между качеством и памятью.

```

In [14]: # Model selection uses train for fit and validation for ranking.
         # The smallest K within 0.01 dB of the best validation NMSE is retained.

```

```

TOPOLOGIES = {
    "общая текущая амплитуда": ((0, 0), (1, 0), (2, 0)),
    "согласованные задержки": ((0, 0), (1, 1), (2, 2)),
}
K_CANDIDATES = (4, 8, 12, 16, 24, 32)
RIDGE = 1e-4
NMSE_TOLERANCE_DB = 0.01
ila_input = train_y / gain
ideal_val = gain * val_x
no_dpd_nmse_db = pooled_nmse_db(pa_val_hat, ideal_val, CASCADE_WARMUP)
selection_rows = []

for topology_name, candidate_branches in TOPOLOGIES.items():
    for candidate_k in K_CANDIDATES:
        candidate_knots = make_quantile_knots(ila_input, candidate_k)
        started = time.perf_counter()
        candidate_coefficients, candidate_warmup = fit_spline_memory_dpd(
            ila_input, train_x, candidate_knots, candidate_branches, RIDGE
        )
        fit_seconds = time.perf_counter() - started
        candidate_drive = predict_spline_memory(
            val_x, candidate_knots, candidate_branches, candidate_coefficients
        )
        candidate_output = gmp_design(candidate_drive, pa_terms) @ pa_coefficients
        selection_rows.append({
            "topology": topology_name,
            "branches": candidate_branches,
            "K": candidate_k,
            "nmse_db": pooled_nmse_db(candidate_output, ideal_val, CASCADE_WARMUP),
            "real_coefficients": 2 * len(candidate_branches) * candidate_k,
            "fit_seconds": fit_seconds,
            "finite": bool(np.all(np.isfinite(candidate_coefficients))),
            "max_drive": float(np.max(np.abs(candidate_drive))),
        })

best_nmse = min(row["nmse_db"] for row in selection_rows)
eligible = [row for row in selection_rows if row["nmse_db"] <= best_nmse + NMSE_TOLERANCE_DB]
selected = min(eligible, key=lambda row: (row["real_coefficients"], row["nmse_db"]))
branches = selected["branches"]
knot_count = selected["K"]
ridge = RIDGE

print(f"no DPD: {no_dpd_nmse_db:.6f} dB")
print(f"selection rule: smallest model within {NMSE_TOLERANCE_DB:.2f} dB of")
print(f"print(f\"{'topology':>29s} {'K':>3s} {'NMSE [dB]':>11s} {'real coef':>10s} {'fit seconds':>9.4f}\"")
for row in selection_rows:
    print(f"{'topology':>29s} {'K':>3s} {'NMSE [dB]':>11s} {'real coef':>10s} {'fit seconds':>9.4f}")
print("selected:", selected)

fig, ax = plt.subplots(figsize=(10.5, 4.6))
for topology_name in TOPOLOGIES:
    rows = [row for row in selection_rows if row["topology"] == topology_name]
    ax.plot([row["K"] for row in rows], [row["nmse_db"] for row in rows],
            marker="o", label=topology_name)
ax.axhline(no_dpd_nmse_db, color="tab:red", ls="--", label="6e3 DPD")

```

```

ax.scatter([knot_count], [selected["nmse_db"]], s=110, facecolors="none",
           edgecolors="black", linewidths=2, label="выбрано")
ax.set(xlabel="число узлов K", ylabel="validation cascade NMSE [dB]",
       title="Выбор топологии и числа узлов (ниже – лучше)")
ax.legend()
plt.tight_layout()
plt.show()

# Refit exactly the chosen configuration on train; deployment receives val_x
knots = make_quantile_knots(ila_input, knot_count)
dpd_coefficients, dpd_fit_warmup = fit_spline_memory_dpd(
    ila_input, train_x, knots, branches, ridge
)
predistorted_val = predict_spline_memory(val_x, knots, branches, dpd_coefficients)
cascade_dpd = gmp_design(predistorted_val, pa_terms) @ pa_coefficients
dpd_metrics = metric_row("зафиксированный spline-memory DPD", cascade_dpd, ideal_val)
no_dpd_mse = np.mean(np.abs(pa_val_hat[CASCADE_WARMUP:] - ideal_val[CASCADE_WARMUP:]))
dpd_mse = np.mean(np.abs(cascade_dpd[CASCADE_WARMUP:] - ideal_val[CASCADE_WARMUP:]))
improvement_db = float(10 * np.log10(no_dpd_mse / dpd_mse))
print("branches:", branches, "knots:", knots.size, "ridge:", ridge)
print("coefficients:", dpd_coefficients.shape, "fit warmup:", dpd_fit_warmup)
print(dpd_metrics)
print(f"improvement over frozen-PA no-DPD: {improvement_db:.3f} dB")

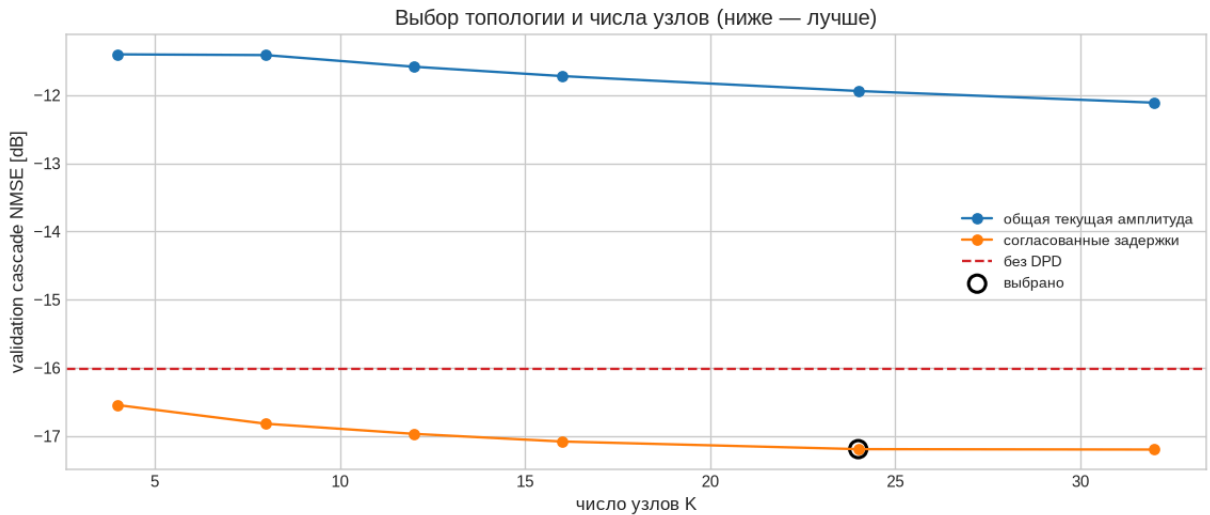
```

no DPD: -16.010351 dB

selection rule: smallest model within 0.01 dB of best

topology	K	NMSE [dB]	real coef	fit [s]
общая текущая амплитуда	4	-11.402039	24	0.1518
общая текущая амплитуда	8	-11.413019	48	0.3823
общая текущая амплитуда	12	-11.583653	72	0.6038
общая текущая амплитуда	16	-11.720803	96	0.8782
общая текущая амплитуда	24	-11.939839	144	1.6488
общая текущая амплитуда	32	-12.111103	192	2.4546
согласованные задержки	4	-16.546080	24	0.1748
согласованные задержки	8	-16.819523	48	0.3536
согласованные задержки	12	-16.968149	72	0.5137
согласованные задержки	16	-17.080311	96	0.8355
согласованные задержки	24	-17.192240	144	1.6485
согласованные задержки	32	-17.198500	192	2.6977

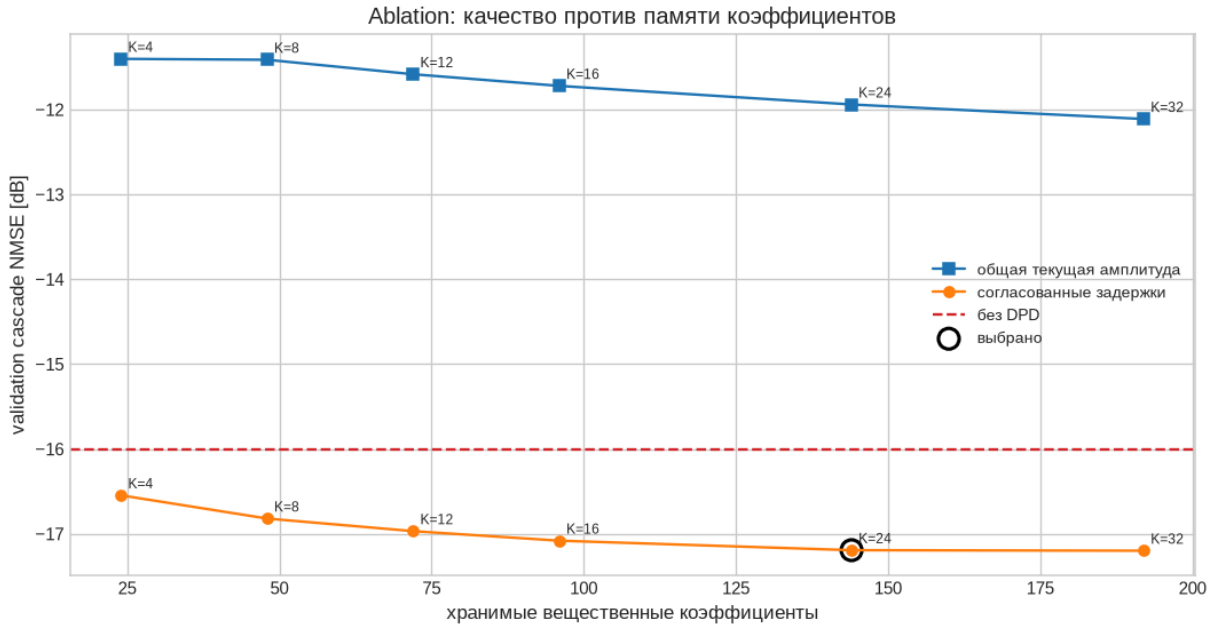
selected: {'topology': 'согласованные задержки', 'branches': ((0, 0), (1, 1), (2, 2)), 'K': 24, 'nmse_db': -17.192240437598432, 'real_coefficients': 144, 'fit_seconds': 1.6484716340000887, 'finite': True, 'max_drive': 0.923164948801733}



branches: ((0, 0), (1, 1), (2, 2)) knots: 24 ridge: 0.0001
 coefficients: (3, 24) fit warmup: 2
 {'name': 'зафиксированный spline-memory DPD', 'nmse_db': -17.192240437598432, 'rms': 0.40311098016628877, 'peak': 1.0222165981698976, 'papr_db': 8.082366012561241}
 improvement over frozen-PA no-DPD: 1.182 dB

```
In [15]: # Дополнительное представление ablation: качество против памяти коэффициентов
fig, ax = plt.subplots(figsize=(9.8, 5.2))
markers = {"общая текущая амплитуда": "s", "согласованные задержки": "o"}
for topology_name in TOPOLOGIES:
    rows = [row for row in selection_rows if row["topology"] == topology_name]
    ax.plot([row["real_coefficients"] for row in rows],
            [row["nmse_db"] for row in rows], marker=markers[topology_name],
            label=topology_name)
    for row in rows:
        ax.annotate(f"K={row['K']}", (row["real_coefficients"], row["nmse_db"]),
                    xytext=(4, 4), textcoords="offset points", fontsize=8)
ax.axhline(no_dpd_nmse_db, color="tab:red", ls="--", label="без DPD")
ax.scatter(selected["real_coefficients"], selected["nmse_db"], s=140,
           facecolors="none", edgecolors="black", linewidths=2, label="выбрано")
ax.set(xlabel="хранимые вещественные коэффициенты",
       ylabel="validation cascade NMSE [dB]",
       title="Ablation: качество против памяти коэффициентов")
ax.legend()
plt.tight_layout()
plt.show()

print("Главный отрицательный результат:")
print("топология с общей текущей амплитудой хуже режима без DPD при всех пробах")
print("Выбранная топология:", selected["topology"], "| K =", selected["K"])
```



Главный отрицательный результат:

топология с общей текущей амплитудой хуже режима без DPD при всех проверенных K .

Выбранная топология: согласованные задержки | $K = 24$

9.1. Direct refinement без изменения fast path

Почему ILA недостаточно

ILA определяет начальный банк $\mathbf{c}_0$ из обратной задачи

$$\min_{\mathbf{c}} \|D_{\mathbf{c}}(y/g) - x\|_2^2,$$

но конечная цель имеет другую функцию потерь:

$$\mathcal{L}(\mathbf{c}) = \|\hat{P}(D_{\mathbf{c}}(x)) - gx\|_2^2.$$

Поэтому ILA используется как устойчивое и быстрое начальное приближение, а не как окончательный оптимум каскада.

Локальная чувствительность каскада

Запишем вещественные и мнимые части 72 комплексных коэффициентов в вектор $\boldsymbol{\theta} \in \mathbb{R}^{144}$. В окрестности текущего банка:

$$\hat{y}(\boldsymbol{\theta} + \Delta\boldsymbol{\theta}) \approx \hat{y}(\boldsymbol{\theta}) + J\Delta\boldsymbol{\theta}.$$

Каждый столбец J вычисляется малым конечным изменением соответствующей вещественной или мнимой части коэффициента:

$$J_k \approx \frac{\hat{P}(D_{\boldsymbol{\theta} + \varepsilon \mathbf{e}_k}(x)) - \hat{P}(D_{\boldsymbol{\theta}}(x))}{\varepsilon}.$$

Такой способ не требует менять PA evaluator и соответствует идее безопасного малого probe для будущего physical closed-loop эксперимента.

Регуляризованное обновление

Для $e = gx - \hat{y}$ решается вещественная ridge-задача

$$\Delta\theta = \arg \min_{\Delta} \|J\Delta - e\|_2^2 + \lambda \|\Delta\|_2^2,$$

после чего

$$\theta_{new} = \theta_{old} + \alpha \Delta\theta.$$

Ridge ограничивает плохо обусловленные направления, а шаг α выбирается на отдельном train-advisor блоке. Jacobian и advisor не пересекаются; validation не используется для коэффициентов, шага или остановки. Число дополнительных итераций зафиксировано заранее. Поскольку меняются только значения LUT, inference formula и operation count остаются прежними.

Что происходит на практике

ILA даёт быструю начальную настройку, но обучает обратное преобразование усилителя, а не весь каскад. Direct refinement задаёт более прямой вопрос: «как небольшое изменение каждого коэффициента изменит ошибку на выходе PA?».

Матрица J , или якобиан, — это просто таблица таких чувствительностей. Ridge-штраф не даёт поправке стать необоснованно большой, а шаг α задаёт, какую долю рассчитанной поправки мы применяем. Всё это выполняется во время калибровки; в рабочем тракте якобиана нет.

```
In [16]: # Воспроизводимый replay финального direct-refined банка и его provenance.
DIRECT_MODEL_B64 = ""UEsDBC0AAAAAIAIQCKsZ+q/////////8SABQAc2NoZWlhX3Zlcr
DIRECT_MODEL_SHA256 = "ddba7f70b63de3c9d3f72991f965a67b7f912795f1cea2556108e
DIRECT_REFINEMENT_JSON_B64 = ""ewogICJzdGFnZSI6ICJkaXJlY3RfcmVmaW5lbWVudCIs
DIRECT_REFINEMENT_JSON_SHA256 = "acd9ebbf2e6ee642f84ab5744668f5164e4e233d45c
ITERATIVE_JSON_B64 = ""ewogICJzdGFnZSI6ICJpdGVyYXRpdmVfYm91bmRlZlZ9kaXJlY3Rf
ITERATIVE_JSON_SHA256 = "22af6417e73effcd2893b0dabfb50fa073e3fa2c76d6d8a053c

def decode_json_payload(encoded, expected_sha256):
    raw_bytes = base64.b64decode(encoded)
    assert hashlib.sha256(raw_bytes).hexdigest() == expected_sha256
    return json.loads(raw_bytes.decode("utf-8"))

model_bytes = base64.b64decode(DIRECT_MODEL_B64)
assert hashlib.sha256(model_bytes).hexdigest() == DIRECT_MODEL_SHA256
with np.load(io.BytesIO(model_bytes), allow_pickle=False) as frozen_model:
    refined_knots = np.asarray(frozen_model["knots"], dtype=float)
    refined_signal_delays = tuple(int(v) for v in frozen_model["signal_delay
    refined_envelope_delays = tuple(int(v) for v in frozen_model["envelope_c
```



```

    refined_dpd_coefficients = np.asarray(
        frozen_model["coefficients"], dtype=np.complex128
    )

direct_report = decode_json_payload(
    DIRECT_REFINEMENT_JSON_B64, DIRECT_REFINEMENT_JSON_SHA256
)
iterative_report = decode_json_payload(ITERATIVE_JSON_B64, ITERATIVE_JSON_SHA256)

assert np.allclose(refined_knots, knots)
assert tuple(zip(refined_signal_delays, refined_envelope_delays)) == tuple(zip(knots, envelope_delays))
assert refined_dpd_coefficients.shape == dpd_coefficients.shape

predistorted_refined_val = predict_spline_memory(
    val_x, refined_knots, branches, refined_dpd_coefficients
)
cascade_refined = gmp_design(predistorted_refined_val, pa_terms) @ pa_coefficients
refined_metrics = metric_row(
    "iterative direct-refined spline", cascade_refined, ideal_val, CASCADE_WARMUP, **kwargs
)
refined_nmse = np.mean(
    np.abs(cascade_refined[CASCADE_WARMUP:] - ideal_val[CASCADE_WARMUP:]) ** 2
)
direct_gain_over_ila_db = dpd_metrics["nmse_db"] - refined_metrics["nmse_db"]
direct_gain_over_no_dpd_db = surrogate_no_dpd["nmse_db"] - refined_metrics["nmse_db"]

# The embedded replay must reproduce the separately generated JSON evidence.
assert abs(refined_metrics["nmse_db"] - iterative_report["final_validation"]["nmse_db"]) < 1e-8
assert abs(dpd_metrics["nmse_db"] - direct_report["baseline_validation"]["nmse_db"]) < 1e-8

print(f"ILA validation NMSE: {dpd_metrics['nmse_db']:.6f} dB")
print(f"Direct-refined NMSE: {refined_metrics['nmse_db']:.6f} dB")
print(f"Gain over ILA: {direct_gain_over_ila_db:.6f} dB")
print(f"Gain over no DPD: {direct_gain_over_no_dpd_db:.6f} dB")
print("Fast-path topology changed: False")
print("Frozen model SHA-256:", DIRECT_MODEL_SHA256)

```

```

ILA validation NMSE: -17.192240 dB
Direct-refined NMSE: -21.587580 dB
Gain over ILA: 4.395340 dB
Gain over no DPD: 5.577229 dB
Fast-path topology changed: False
Frozen model SHA-256: ddba7f70b63de3c9d3f72991f965a67b7f912795f1cea2556108e1a31a8eb222

```

10. Основной результат на validation

Здесь сравниваются не две разные DPD-архитектуры, а последовательные состояния **одной spline-memory модели**:

1. PA без DPD;
2. наш spline-memory baseline после начальной ILA-калибровки;

3. тот же baseline после direct refinement его 72 комплексных коэффициентов.

Число ветвей, узлов, задержек и операций в пунктах 2 и 3 одинаково. Поэтому разность качества показывает эффект способа калибровки, а не выигрыш от более крупной модели. Ось NMSE сфокусирована на рабочем диапазоне dB; над каждым столбцом приведено точное значение.

```
In [17]: # NMSE, сходимость уточнения и уровни сигнала перед PA.
NO_DPD_COLOR = "0.55"
BASELINE_COLOR = "#FDB863"
REFINED_COLOR = "#E66101"

result_names = ["без DPD", "наш baseline\nпосле ILA", "тот же baseline\nпосле"]
result_nmse = [surrogate_no_dpd["nmse_db"], dpd_metrics["nmse_db"],
               refined_metrics["nmse_db"]]

history = [dpd_metrics["nmse_db"],
           iterative_report["one_update_validation"]["nmse_db"]]
history.extend(
    row["validation_diagnostic_not_used_for_stopping"]["nmse_db"]
    for row in iterative_report["schedules"]["predeclared_A"]["updates"]
)
history_labels = ["ILA baseline", "+1", "+2", "+3", "+4"]

fig, axes = plt.subplots(1, 3, figsize=(17, 4.8))
bars = axes[0].bar(result_names, result_nmse,
                  color=[NO_DPD_COLOR, BASELINE_COLOR, REFINED_COLOR])
axes[0].set_ylabel="pooled complex NMSE [dB]",
          title="Одна архитектура до и после уточнения")
axes[0].set_ylim(min(result_nmse) - 1.0, max(result_nmse) + 1.0)
for bar, value in zip(bars, result_nmse):
    axes[0].text(bar.get_x() + bar.get_width()/2, value + 0.18,
                 f"{value:.3f}", ha="center", weight="bold")

axes[1].plot(np.arange(len(history)), history, "o-", color=REFINED_COLOR)
axes[1].set_xticks(np.arange(len(history)), history_labels, rotation=18)
axes[1].set_ylabel="validation NMSE [dB]",
          title="Уточнение коэффициентов baseline")
for index, value in enumerate(history):
    axes[1].annotate(f"{value:.3f}", (index, value), xytext=(0, 7),
                    textcoords="offset points", ha="center", fontsize=8)

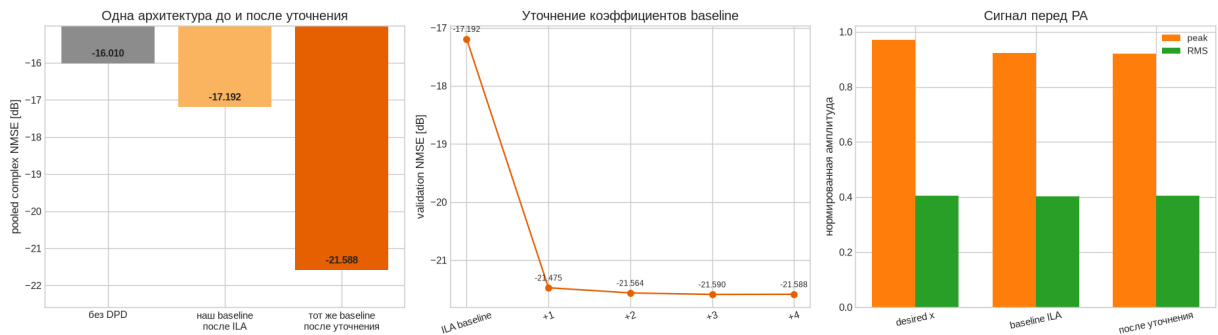
drive_rows = {
    "desired x": summary(val_x[CASCADE_WARMUP:]),
    "baseline ILA": summary(predistorted_val[CASCADE_WARMUP:]),
    "после уточнения": summary(predistorted_refined_val[CASCADE_WARMUP:]),
}
drive_names = list(drive_rows)
positions = np.arange(len(drive_names))
axes[2].bar(positions - 0.18,
            [drive_rows[name]["peak"] for name in drive_names], width=0.36,
            label="peak", color="tab:orange")
axes[2].bar(positions + 0.18,
            [drive_rows[name]["rms"] for name in drive_names], width=0.36,
```

```

label="RMS", color="tab:green")
axes[2].set_xticks(positions, drive_names, rotation=12)
axes[2].set(title="Сигнал перед PA", ylabel="нормированная амплитуда")
axes[2].legend()
plt.tight_layout()
plt.show()

print(f"Выигрыш уточнения относительно ILA-baseline: {direct_gain_over_ila_db}")
print(f"Выигрыш уточнённого baseline относительно режима без DPD: {direct_gain_over_dpd_db}")
print("Baseline после ILA:", drive_rows["baseline ILA"])
print("Тот же baseline после уточнения:", drive_rows["после уточнения"])

```



Выигрыш уточнения относительно ILA-baseline: 4.395340 dB

Выигрыш уточнённого baseline относительно режима без DPD: 5.577229 dB

Baseline после ILA: {'rms': 0.402828007934089, 'peak': 0.923164948801733, 'prapr_db': 7.20319295870158}

Тот же baseline после уточнения: {'rms': 0.40488424160330644, 'peak': 0.920485351521888, 'prapr_db': 7.133720151196402}

10.1. AM/AM и AM/PM до и после коррекции

Этот рисунок наглядно показывает два типа искажений. **AM/AM** отвечает на вопрос, как меняется выходная амплитуда при росте $|x|$. **AM/PM** показывает ошибку фазы:

$$\Delta\varphi[n] = \arg(\hat{y}[n](gx[n])^*).$$

Верхний ряд — PA evaluator без DPD. Нижний ряд — наш spline-memory DPD: жёлтым показана начальная ILA-калибровка, оранжевым — та же модель после direct refinement. Чёрная линия соответствует идеальному ответу. Точки не являются отдельными I/Q-моделями: они рассчитаны из единого комплексного сигнала.

У очень слабых отсчётов фаза плохо определена, поэтому в AM/PM не показаны точки ниже 2% максимальной амплитуды. Полоса 10–90% показывает разброс большинства отсчётов, а не доверительный интервал.

```

In [18]: # AM/AM and AM/PM before and after correction on the same validation record.
radius = np.abs(val_x)
ideal_amplitude = np.abs(ideal_val)
sample_indices = np.arange(CASCADE_WARMUP, val_x.size, 25)
r_grid = np.linspace(0, radius.max(), 300)

```

```

def phase_error_degrees(output, reference):
    """Wrapped phase difference between complex output and desired reference
    return np.degrees(np.angle(output * np.conj(reference)))

phase_mask = ideal_amplitude > 0.02 * np.max(ideal_amplitude)
phase_indices = np.flatnonzero(phase_mask)[:25]
phase_no_dpd = phase_error_degrees(pa_val_hat, ideal_val)
phase_ila = phase_error_degrees(cascade_dpd, ideal_val)
phase_direct = phase_error_degrees(cascade_refined, ideal_val)

no_dpd_am = quantile_binned_statistics(radius, np.abs(pa_val_hat), 32)
ila_am = quantile_binned_statistics(radius, np.abs(cascade_dpd), 32)
direct_am = quantile_binned_statistics(radius, np.abs(cascade_refined), 32)
no_dpd_pm = quantile_binned_statistics(radius[phase_mask], phase_no_dpd[phase_mask])
ila_pm = quantile_binned_statistics(radius[phase_mask], phase_ila[phase_mask])
direct_pm = quantile_binned_statistics(radius[phase_mask], phase_direct[phase_mask])

fig, axes = plt.subplots(2, 2, figsize=(15.5, 9.4), sharex="col")

axes[0, 0].scatter(radius[sample_indices], np.abs(pa_val_hat)[sample_indices],
                  s=3, alpha=0.12, color=NO_DPD_COLOR, label="отдельные отсчеты")
axes[0, 0].plot(no_dpd_am["center"], no_dpd_am["median"],
               color=NO_DPD_COLOR, lw=2.0, label="медиана без DPD")
axes[0, 0].plot(r_grid, abs(gain) * r_grid, "k--", lw=1.5, label="идеал |g|")
axes[0, 0].set(ylabel="|выход PA|", title="AM/AM до коррекции")
axes[0, 0].legend(fontsize=8)

axes[0, 1].scatter(radius[phase_indices], phase_no_dpd[phase_indices],
                  s=3, alpha=0.12, color=NO_DPD_COLOR, label="отдельные отсчеты")
axes[0, 1].plot(no_dpd_pm["center"], no_dpd_pm["median"],
               color=NO_DPD_COLOR, lw=2.0, label="медиана без DPD")
axes[0, 1].axhline(0.0, color="black", ls="--", lw=1.2, label="идеал: 0°")
axes[0, 1].set(ylabel="фазовая ошибка, градусы", title="AM/PM до коррекции",
               legend=axes[0, 0].legend())
axes[0, 1].legend(fontsize=8)

axes[1, 0].plot(ila_am["center"], ila_am["median"],
               color=BASELINE_COLOR, ls="--", lw=2.2, label="baseline после коррекции")
axes[1, 0].scatter(radius[sample_indices], np.abs(cascade_refined)[sample_indices],
                  s=3, alpha=0.15, color=REFINED_COLOR, label="direct: отдельные отсчеты")
axes[1, 0].fill_between(direct_am["center"], direct_am["q10"], direct_am["q90"],
                      color=REFINED_COLOR, alpha=0.18, label="direct: 10–90%")
axes[1, 0].plot(direct_am["center"], direct_am["median"],
               color=REFINED_COLOR, lw=2.2, label="после direct refinement")
axes[1, 0].plot(r_grid, abs(gain) * r_grid, "k--", lw=1.5, label="идеал |g|")
axes[1, 0].set(xlabel="желаемая амплитуда |x|", ylabel="|выход PA|",
               title="AM/AM после коррекции")
axes[1, 0].legend(fontsize=8, ncol=2)

axes[1, 1].plot(ila_pm["center"], ila_pm["median"],
               color=BASELINE_COLOR, ls="--", lw=2.2, label="baseline после коррекции")
axes[1, 1].scatter(radius[phase_indices], phase_direct[phase_indices],
                  s=3, alpha=0.15, color=REFINED_COLOR, label="direct: отдельные отсчеты")
axes[1, 1].fill_between(direct_pm["center"], direct_pm["q10"], direct_pm["q90"],
                      color=REFINED_COLOR, alpha=0.18, label="direct: 10–90%")

```

```

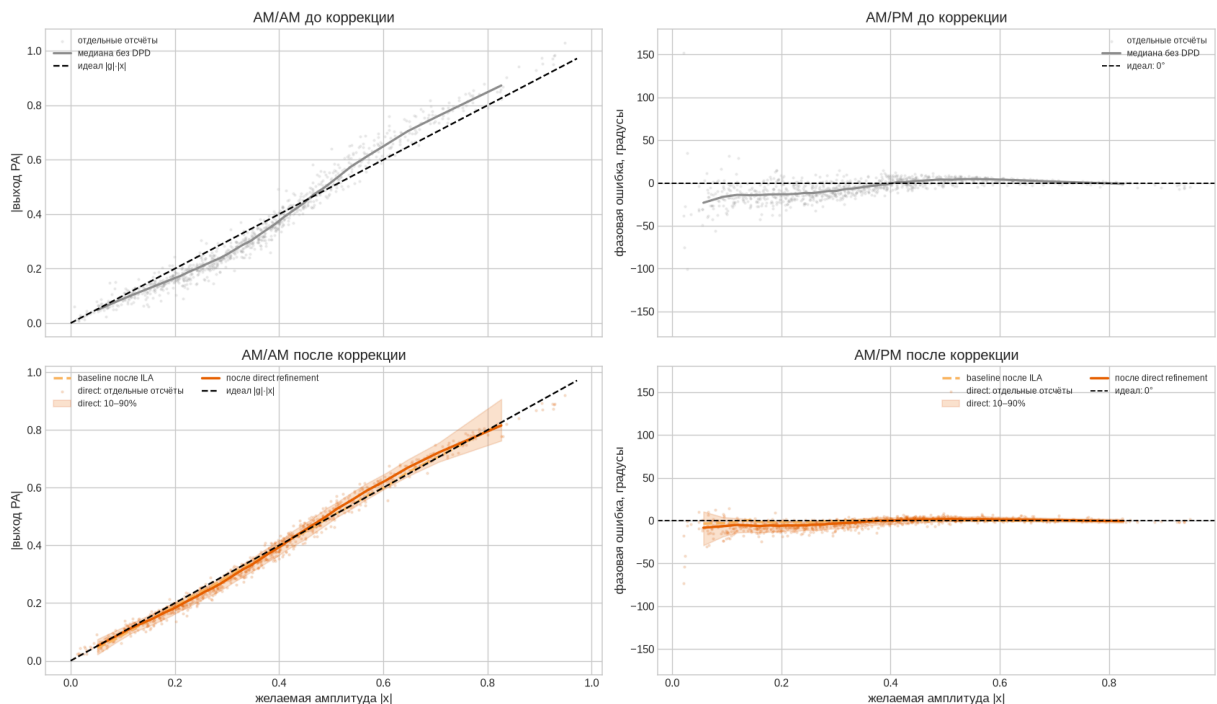
color=REFINED_COLOR, alpha=0.18, label="direct: 10-90%")
axes[1, 1].plot(direct_pm["center"], direct_pm["median"],
                color=REFINED_COLOR, lw=2.2, label="после direct refinement")
axes[1, 1].axhline(0.0, color="black", ls="--", lw=1.2, label="идеал: 0°")
axes[1, 1].set(xlabel="желаемая амплитуда |x|", ylabel="фазовая ошибка, град",
               title="AM/PM после коррекции", ylim=(-180, 180))
axes[1, 1].legend(fontsize=8, ncol=2)

fig.suptitle("Амплитудные и фазовые искажения до и после DPD", fontsize=15,
plt.tight_layout()
plt.show()

print("Оранжевый результат – та же spline-memory архитектура после direct refinement")
print("Измеренный validation y не подавался на вход DPD.")

```

Амплитудные и фазовые искажения до и после DPD



Оранжевый результат – та же spline-memory архитектура после direct refinement.

Измеренный validation y не подавался на вход DPD.

10.2 Коэффициенты одной модели до и после улучшения

На каждом графике одновременно показана **одна spline-memory архитектура** в двух состояниях:

- жёлтая пунктирная линия и круглые узлы — исходный baseline после ILA;
- оранжевая линия и квадратные узлы — тот же baseline после direct refinement.

Одинаковые оси позволяют непосредственно увидеть изменение каждого комплексного коэффициента. Точки являются узлами LUT, линии — линейной

интерполяцией рабочего тракта. Под рисунком приведены численные значения representative knots; полная таблица всех 24 узлов раскрывается по нажатию.

```
In [19]: # Два банка одной архитектуры: ILA-начало и тот же baseline после уточнения.
from html import escape
from IPython.display import HTML, display

dense_radius = np.linspace(knots[0], knots[-1], 500)
dense_left, dense_weight = spline_coordinates(dense_radius, knots)
component_names = ("Re C_m(r)", "Im C_m(r)", "|C_m(r)|",
                  "phase C_m(r) [градусы]")

def coefficient_components(coefficients):
    """Return dense real, imaginary, magnitude and phase curves by branch."""
    result = []
    for branch_coefficients in coefficients:
        dense = (branch_coefficients[dense_left]
                 + dense_weight * (branch_coefficients[dense_left + 1]
                                   - branch_coefficients[dense_left]))
        result.append((dense.real, dense.imag, np.abs(dense),
                      np.degrees(np.angle(dense))))
    return result

ila_components = coefficient_components(dpd_coefficients)
refined_components = coefficient_components(refined_dpd_coefficients)

# Одинаковые пределы соответствующих осей для честного сравнения двух банков
component_limits = {}
for branch_index in range(len(branches)):
    for column in range(4):
        combined = np.concatenate((ila_components[branch_index][column],
                                   refined_components[branch_index][column]))
        lower, upper = float(np.min(combined)), float(np.max(combined))
        margin = 0.08 * max(upper - lower, 1e-3)
        component_limits[branch_index, column] = (lower - margin, upper + margin)

def plot_combined_coefficient_banks():
    """Overlay ILA and refined control points on identical axes."""
    fig, axes = plt.subplots(3, 4, figsize=(16, 10), sharex=True)
    for branch_index, (signal_delay, envelope_delay) in enumerate(branches):
        ila_knots = dpd_coefficients[branch_index]
        refined_knots = refined_dpd_coefficients[branch_index]
        ila_knot_values = (
            ila_knots.real,
            ila_knots.imag,
            np.abs(ila_knots),
            np.degrees(np.angle(ila_knots)),
        )
        refined_knot_values = (
            refined_knots.real,
            refined_knots.imag,
```

```

        np.abs(refined_knots),
        np.degrees(np.angle(refined_knots)),
    )
    for column, ylabel in enumerate(component_names):
        axis = axes[branch_index, column]
        axis.plot(
            dense_radius,
            ila_components[branch_index][column],
            color=BASELINE_COLOR,
            ls="--",
            lw=2.0,
            label="baseline после ILA" if branch_index == column == 0 else None
        )
        axis.plot(
            knots,
            ila_knot_values[column],
            "o",
            ms=3.2,
            color=BASELINE_COLOR,
            alpha=0.90,
            label="узлы ILA" if branch_index == column == 0 else None,
        )
        axis.plot(
            dense_radius,
            refined_components[branch_index][column],
            color=REFINED_COLOR,
            lw=1.9,
            label="тот же baseline после refinement"
            if branch_index == column == 0 else None,
        )
        axis.plot(
            knots,
            refined_knot_values[column],
            "s",
            ms=3.0,
            color=REFINED_COLOR,
            alpha=0.90,
            label="узлы refined" if branch_index == column == 0 else None
        )
        axis.set_ylim(*component_limits[branch_index, column])
        axis.set_ylabel(ylabel)
        axis.set_title(
            f"C{branch_index}: x[n-{signal_delay}], |x[n-{envelope_delay}]|
        )
        if branch_index == len(branches) - 1:
            axis.set_xlabel("нормированная амплитуда r")
    axes[0, 0].legend(fontsize=7.5)
    fig.suptitle(
        "Один spline-memory baseline: ILA и результат direct refinement",
        y=0.995,
        weight="bold",
    )
    plt.tight_layout(rect=(0, 0, 1, 0.97))
    plt.show()

```

```

plot_combined_coefficient_banks()

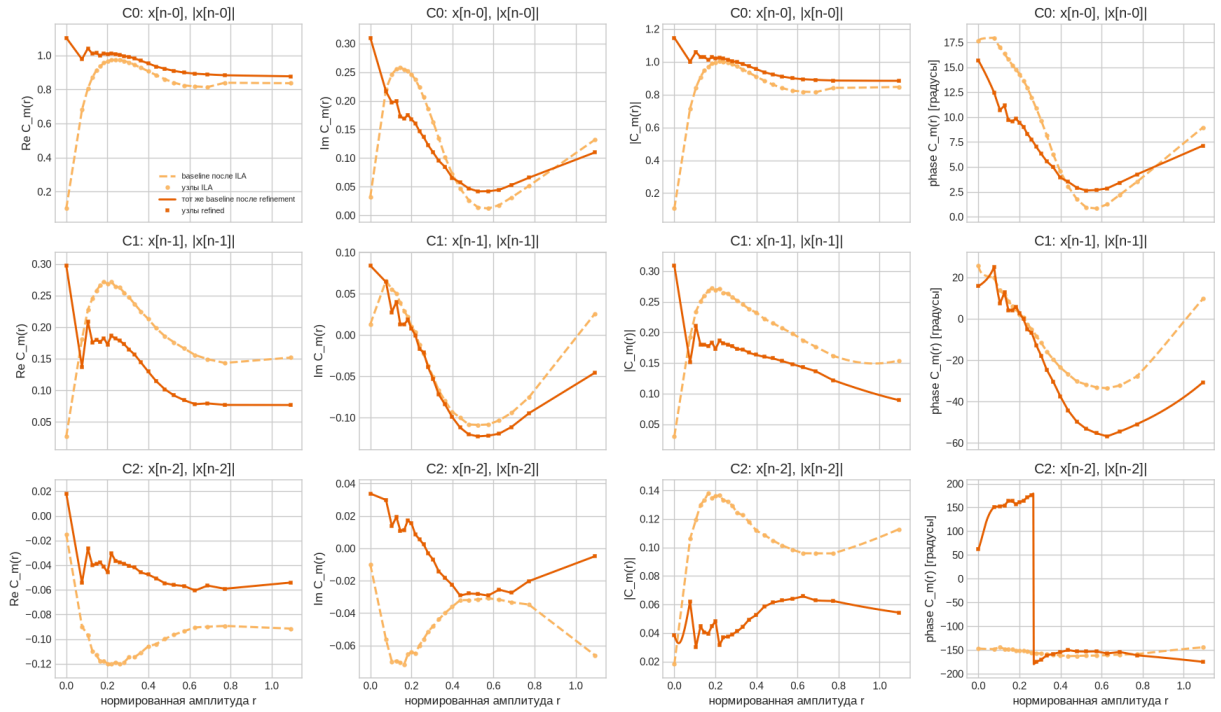
def format_complex(value):
    """Format one complex control point with explicit real and imaginary part"""
    return f"{value.real:+.6f}{value.imag:+.6f}j"

def coefficient_table_html(indices, title, opened=False):
    """Build a compact exact-value table for selected or all spline knots."""
    rows = []
    for knot_index in indices:
        cells = [str(knot_index), f"{knots[knot_index]:.6f}"]
        for branch_index in range(len(branches)):
            cells.extend((
                format_complex(dpd_coefficients[branch_index, knot_index]),
                format_complex(refined_dpd_coefficients[branch_index, knot_index])
            ))
        rows.append("<tr>" + "".join(f"<td>{escape(value)}</td>" for value in cells))
    headers = ["k", "r"]
    for branch_index in range(len(branches)):
        headers.extend((f"C{branch_index} ILA", f"C{branch_index} refined"))
    table = (
        "<table style='border-collapse:collapse;font-family:monospace;font-size:10px;'>"
        "<thead><tr>" + "".join(
            f"<th style='padding:4px 8px;border:1px solid #bbb'>{escape(header)}"
            for header in headers
        ) + "</tr></thead><tbody>" + "".join(rows) + "</tbody></table>"
    )
    if opened:
        return f"<h4>{escape(title)}</h4>{table}"
    return f"<details><summary><b>{escape(title)}</b></summary>{table}</details>"

representative_indices = np.unique(
    np.round(np.linspace(0, knot_count - 1, 7)).astype(int)
)
display(HTML(coefficient_table_html(
    representative_indices,
    "Численные значения representative knots: ILA и refined",
    opened=True,
)))
display(HTML(coefficient_table_html(
    np.arange(knot_count),
    "Показать точные значения всех 24 узлов",
    opened=False,
)))

coefficient_relative_change = float(
    np.linalg.norm(refined_dpd_coefficients - dpd_coefficients)
    / np.linalg.norm(dpd_coefficients)
)
print(f"Относительное изменение одного и того же банка: {coefficient_relative_change}")
print("Topology, K и стоимость inference не изменились.")

```

Численные значения representative knots: ILA и refined

k	r	C0 ILA	C0 refined	C1 ILA	C1 refined
0	0.000000	+0.101324+0.032273j	+1.103434+0.310031j	+0.027251+0.013114j	+0.297234+0.013114j
4	0.146192	+0.912266+0.258281j	+1.013815+0.172997j	+0.257324+0.038828j	+0.179868+0.038828j
8	0.219585	+0.974664+0.237130j	+1.010468+0.160544j	+0.271533+0.003542j	+0.186183+0.003542j
12	0.305986	+0.959300+0.163156j	+0.992244+0.109730j	+0.247062-0.050490j	+0.164606-0.050490j
15	0.398598	+0.907063+0.072437j	+0.953455+0.065543j	+0.213211-0.092697j	+0.129831-0.092697j
19	0.573915	+0.823970+0.011893j	+0.898813+0.041968j	+0.166222-0.108349j	+0.084407-0.108349j
23	1.094298	+0.836562+0.132076j	+0.877142+0.109895j	+0.151833+0.026005j	+0.076807+0.026005j



► Показать точные значения всех 24 узлов

Относительное изменение одного и того же банка: 0.292415
Topology, K и стоимость inference не изменились.

10.3 Разложение одного отсчёта

Следующий рисунок делает работу трёх ветвей наблюдаемой. Для выбранного n вычисляются

$$u_m[n] = x[n - m]C_m(|x[n - m]|), \quad z[n] = u_0[n] + u_1[n] + u_2[n].$$

Векторы на комплексной плоскости складываются последовательно.

```
In [20]: # Один отсчёт финального direct-refined DPD: разложение z[n] на три комплексных вектора
example_n = CASCADE_WARMUP + 137
contributions = []
example_values = []
for branch_index, (signal_delay, envelope_delay) in enumerate(branches):
    radius = abs(val_x[example_n - envelope_delay])
    left, weight = spline_coordinates(np.array([radius]), knots)
    c_value = (refined_dpd_coefficients[branch_index, left[0]]
               + weight[0] * (refined_dpd_coefficients[branch_index, left[1]]
                              - refined_dpd_coefficients[branch_index, left[2]]))
    contribution = val_x[example_n - signal_delay] * c_value
    contributions.append(contribution)
    example_values.append((radius, c_value))
z_example = sum(contributions)
assert abs(z_example - predistorted_refined_val[example_n]) < 1e-12

print(f"n = {example_n}")
for index, ((radius, c_value), contribution) in enumerate(zip(example_values, contributions)):
    print(f"u{index}: r={radius:.6f}, C{index}(r)={c_value:.6f}, u{index}={contribution}")
print(f"z[n] = u0+u1+u2 = {z_example:.6f}")

fig, ax = plt.subplots(figsize=(6.4, 6.0))
origin = 0j
for index, contribution in enumerate(contributions):
    ax.quiver(origin.real, origin.imag, contribution.real, contribution.imag,
              angles="xy", scale_units="xy", scale=1, width=0.008,
              color=f"C{index}", label=f"u{index}")
    origin += contribution
ax.scatter([z_example.real], [z_example.imag], color="black", s=45, label="z[n]")
extent = 1.2 * max(abs(z_example), *(abs(value) for value in contributions))
ax.set(xlim=(-extent, extent), ylim=(-extent, extent), xlabel="вещественная часть",
       ylabel="мнимая часть", title="Сумма ветвей для одного отсчёта")
ax.set_aspect("equal", adjustable="box")
ax.legend()
plt.tight_layout()
plt.show()
```

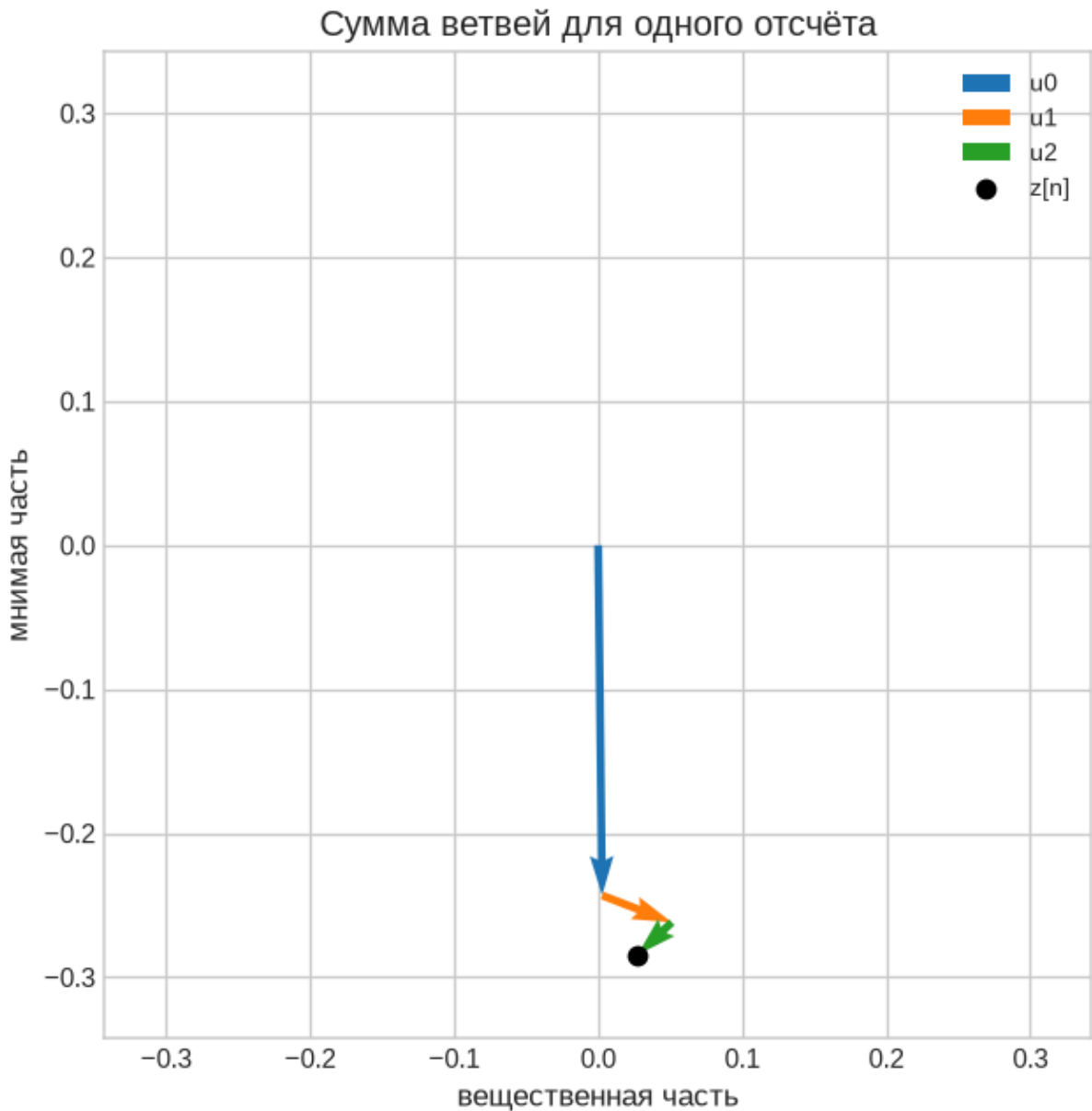
$n = 148$

u_0 : $r=0.238663$, $C_0(r)=1.008381+0.147294j$, $u_0=0.002723-0.243202j$

u_1 : $r=0.297460$, $C_1(r)=0.167539-0.048139j$, $u_1=0.048379-0.018659j$

u_2 : $r=0.520994$, $C_2(r)=-0.056009-0.028098j$, $u_2=-0.023702-0.022450j$

$z[n] = u_0+u_1+u_2 = 0.027400-0.284311j$



10.4 Фрагмент во времени

Показан один и тот же 600-sample интервал. Верхняя панель сравнивает выход каскада с gx ; нижняя показывает мгновенный модуль комплексной ошибки и локальный RMS. Толстая RMS-линия нужна, потому что знакопеременная вещественная часть ошибки визуальнo скрывает её мощность.

```
In [21]: # 600 отсчётов: финальный direct-refined каскад и локальная ошибка.
fragment_length = 600
window = slice(CASCADE_WARMUP, CASCADE_WARMUP + fragment_length)
sample_axis = np.arange(fragment_length) + CASCADE_WARMUP
fig, axes = plt.subplots(2, 1, figsize=(15.5, 7.8), sharex=True)
axes[0].plot(sample_axis, ideal_val[window].real, color="black", lw=1.15,
              label="ideal g·x")
axes[0].plot(sample_axis, pa_val_hat[window].real, color="tab:red",
              alpha=0.55, lw=0.7, label="PA без DPD")
```

```

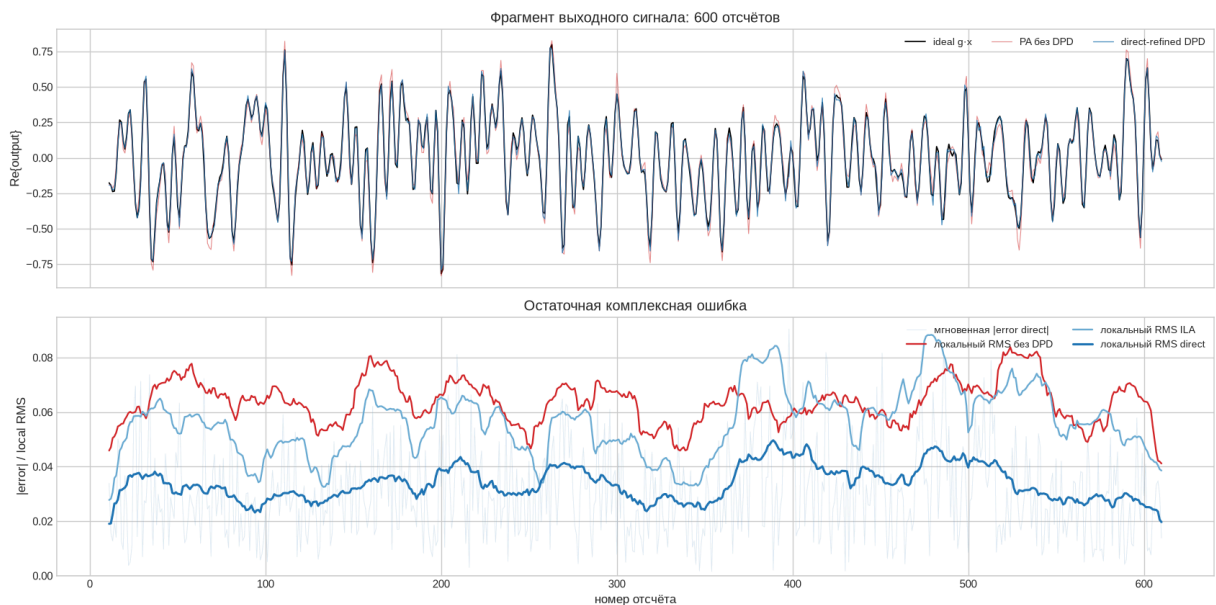
axes[0].plot(sample_axis, cascade_refined[window].real, color="tab:blue",
             alpha=0.82, lw=0.8, label="direct-refined DPD")
axes[0].set_ylabel("Re{output}")
axes[0].set_title("Фрагмент выходного сигнала: 600 отсчётов")
axes[0].legend(loc="upper right", ncols=3)

error_no_dpd = (pa_val_hat - ideal_val)[window]
error_ila = (cascade_dpd - ideal_val)[window]
error_direct = (cascade_refined - ideal_val)[window]

def local_rms(signal: np.ndarray, width=25) -> np.ndarray:
    kernel = np.ones(width, dtype=float) / width
    return np.sqrt(np.convolve(np.abs(signal) ** 2, kernel, mode="same"))

axes[1].plot(sample_axis, np.abs(error_direct), color="tab:blue",
             lw=0.5, alpha=0.15, label="мгновенная |error direct|")
axes[1].plot(sample_axis, local_rms(error_no_dpd), color="tab:red", lw=1.5,
             label="локальный RMS без DPD")
axes[1].plot(sample_axis, local_rms(error_ila), color="#6BAED6", lw=1.5,
             label="локальный RMS ILA")
axes[1].plot(sample_axis, local_rms(error_direct), color="tab:blue", lw=2.0,
             label="локальный RMS direct")
axes[1].set_xlabel="номер отсчёта", ylabel="|error| / local RMS",
             title="Остаточная комплексная ошибка")
axes[1].set_ylim(bottom=0)
axes[1].legend(loc="upper right", ncols=2)
plt.tight_layout()
plt.show()

```



10.5 Устойчивость результата по временным сегментам

Pooled NMSE может скрыть локальное ухудшение. Поэтому validation дополнительно делится на восемь последовательных сегментов без перемешивания. Это диагностическая проверка, а не восемь независимых экспериментов.

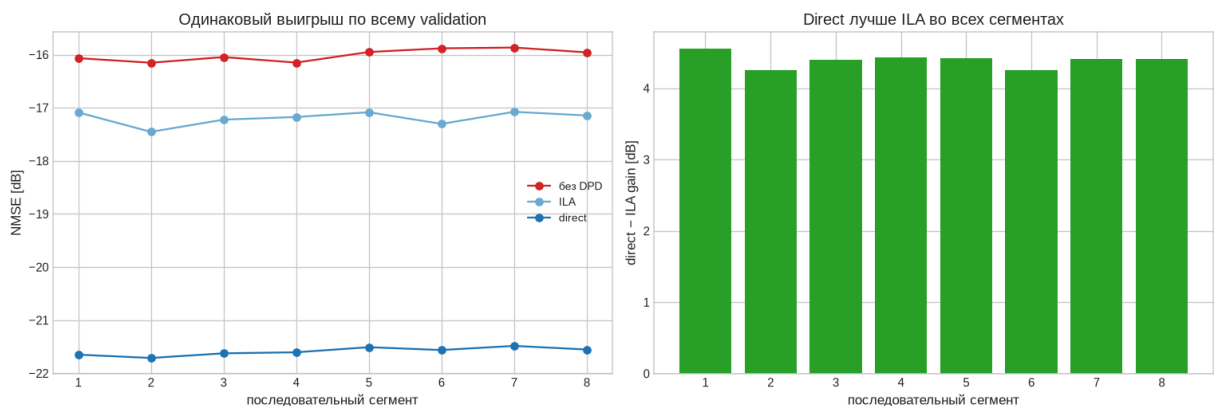
```

In [22]: # Segment-wise устойчивость ILA и direct refinement.
segment_count = 8
indices = np.arange(CASCADE_WARMUP, val_x.size)
segments = np.array_split(indices, segment_count)
segment_no = np.asarray([
    pooled_nmse_db(pa_val_hat[s], ideal_val[s]) for s in segments
])
segment_ila = np.asarray([
    pooled_nmse_db(cascade_dpd[s], ideal_val[s]) for s in segments
])
segment_direct = np.asarray([
    pooled_nmse_db(cascade_refined[s], ideal_val[s]) for s in segments
])
segment_gain_no_to_direct = segment_no - segment_direct
segment_gain_ila_to_direct = segment_ila - segment_direct

fig, axes = plt.subplots(1, 2, figsize=(14, 4.8))
x_segments = np.arange(1, segment_count + 1)
axes[0].plot(x_segments, segment_no, "o-", color="tab:red", label="без DPD")
axes[0].plot(x_segments, segment_ila, "o-", color="#6BAED6", label="ILA")
axes[0].plot(x_segments, segment_direct, "o-", color="tab:blue", label="direct")
axes[0].set(xlabel="последовательный сегмент", ylabel="NMSE [dB]",
            title="Одинаковый выигрыш по всему validation")
axes[0].legend()
axes[1].bar(x_segments, segment_gain_ila_to_direct, color="tab:green")
axes[1].axhline(0, color="black", lw=0.8)
axes[1].set(xlabel="последовательный сегмент", ylabel="direct - ILA gain [dB]",
            title="Direct лучше ILA во всех сегментах")
plt.tight_layout()
plt.show()

print("Direct gain over ILA by segment [dB]:",
      np.round(segment_gain_ila_to_direct, 4))
print("Improved segments:", int(np.sum(segment_gain_ila_to_direct > 0)), "/"

```



Direct gain over ILA by segment [dB]: [4.5608 4.2577 4.3999 4.4304 4.4238 4.258 4.407 4.4071]

Improved segments: 8 / 8

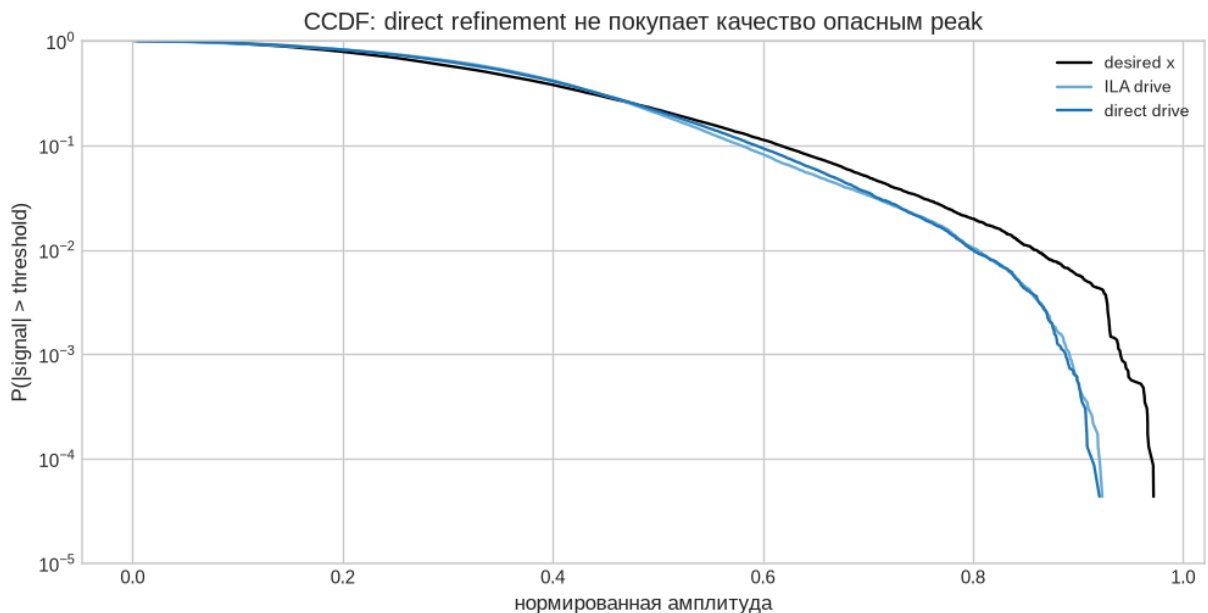
10.6 Амплитудный запас и PAPR

Линеаризация не должна достигаться неограниченным ростом drive. Помимо peak и RMS полезно сравнить эмпирическую CCDF амплитуды desired и predistorted сигналов.

```
In [23]: def empirical_ccdf(values: np.ndarray):
    ordered = np.sort(np.asarray(values, dtype=float))
    exceedance = (ordered.size - np.arange(ordered.size)) / ordered.size
    return ordered, exceedance

fig, ax = plt.subplots(figsize=(9.2, 4.8))
for label, signal, color in [
    ("desired x", val_x[CASCADE_WARMUP:], "black"),
    ("ILA drive", predistorted_val[CASCADE_WARMUP:], "#6BAED6"),
    ("direct drive", predistorted_refined_val[CASCADE_WARMUP:], "tab:blue"),
]:
    amplitude, probability = empirical_ccdf(np.abs(signal))
    ax.semilogy(amplitude, probability, color=color, label=label)
ax.set(xlabel="нормированная амплитуда", ylabel="P(|signal| > threshold)",
      title="CCDF: direct refinement не покупает качество опасным peak",
      ylim=(1e-5, 1.0))
ax.legend()
plt.tight_layout()
plt.show()

print(f"desired PAPR: {summary(val_x[CASCADE_WARMUP:]['papr_db']:.4f) dB")
print(f"ILA PAPR:      {summary(predistorted_val[CASCADE_WARMUP:]['papr_db']:.4f) dB")
print(f"direct PAPR:    {summary(predistorted_refined_val[CASCADE_WARMUP:]['papr_db']:.4f) dB")
print(f"maximum |z_direct|: {np.max(np.abs(predistorted_refined_val)):.6f}")
```



```
desired PAPR: 7.6172 dB
ILA PAPR:      7.2032 dB
direct PAPR:   7.1337 dB
maximum |z_direct|: 0.920485
```

11. Спектральная диагностика

PSD оценивается методом Welch: Hann window, `N=2048`, `overlap 1536`, затем выполняется небольшое сглаживание **линейной мощности** только для читаемости рисунка.

Для BlackBox неизвестны частота дискретизации и официальные границы основной/соседних полос. Поэтому ось показывается в циклах на отсчёт, а произвольный ACLR не придумывается. Чёрный пунктир — реальный спектр g_x , который не обязан быть плоским.

Как читать спектр

Центральная широкая часть — полезный сигнал. «Плечи» слева и справа появляются из-за нелинейности РА. Чем ближе крива DPD к чёрному идеалу вне центральной области, тем меньше спектральное разрастание. График ниже вынесен отдельно, чтобы различия не сжимались в узкой панели.

```
In [24]: def welch_complex(signal: np.ndarray, nperseg=2048, noverlap=1536):
    """Estimate a normalized-frequency Welch PSD for a complex vector."""
    signal = np.asarray(signal, dtype=np.complex128)
    step = nperseg - noverlap
    window = np.hanning(nperseg)
    scale = np.sum(window ** 2)
    starts = range(0, signal.size - nperseg + 1, step)
    spectra = []
    for start in starts:
        segment = signal[start:start + nperseg]
        spectra.append(np.abs(np.fft.fftshift(np.fft.fft(segment * window))))
    if not spectra:
        raise ValueError("signal is shorter than nperseg")
    power = np.mean(np.stack(spectra), axis=0)
    frequency = np.fft.fftshift(np.fft.fftfreq(nperseg, d=1.0))
    return frequency, power

def smooth_linear_power(power: np.ndarray, bins=17) -> np.ndarray:
    """Smooth PSD samples in linear power with an odd Hann kernel."""
    power = np.asarray(power, dtype=float)
    bins = int(bins)
    if bins < 3:
        return power.copy()
    if bins % 2 == 0:
        bins += 1
    kernel = np.hanning(bins)
    kernel /= np.sum(kernel)
    pad = bins // 2
    padded = np.pad(power, (pad, pad), mode="edge")
    return np.convolve(padded, kernel, mode="valid")

# A shorter segment gives more Welch averages on the 23k-sample
# validation set; the mild frequency smoothing is presentation-only.
```

```

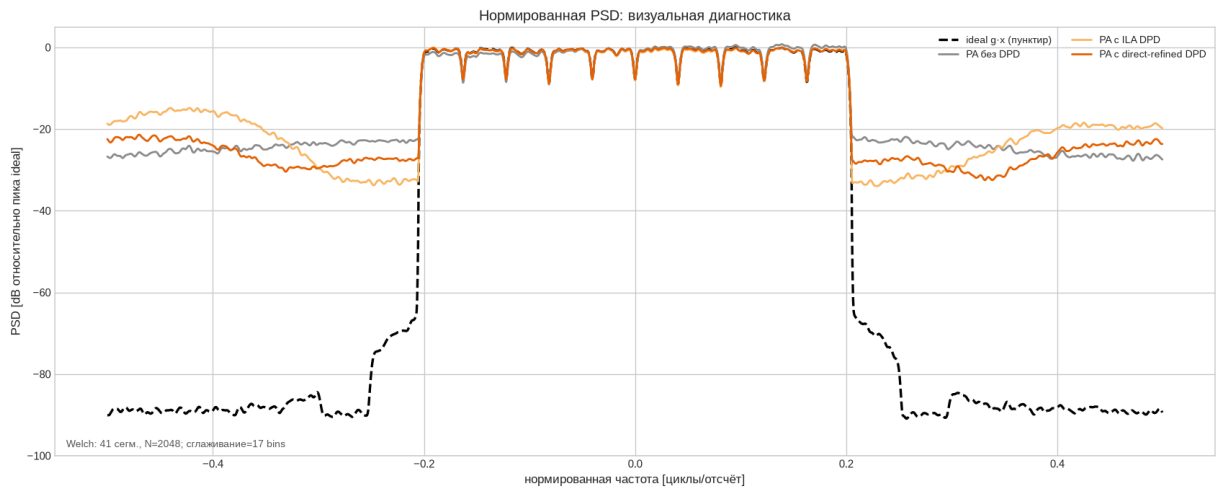
PSD_NPERSEG = 2048
PSD_NOVERLAP = 1536
PSD_SMOOTH_BINS = 17
psd_signals = {
    "ideal": ideal_val[CASCADE_WARMUP:],
    "no DPD": pa_val_hat[CASCADE_WARMUP:],
    "ILA DPD": cascade_dpd[CASCADE_WARMUP:],
    "direct DPD": cascade_refined[CASCADE_WARMUP:],
}
psd = {
    name: welch_complex(signal, PSD_NPERSEG, PSD_NOVERLAP)
    for name, signal in psd_signals.items()
}
psd_smooth = {
    name: (frequency, smooth_linear_power(power, PSD_SMOOTH_BINS))
    for name, (frequency, power) in psd.items()
}
reference_peak = np.max(psd_smooth["ideal"][1])

colors = {"ideal": "black", "no DPD": NO_DPD_COLOR, "ILA DPD": BASELINE_COLOR}
labels = {
    "ideal": "ideal g·x (пунктир)",
    "no DPD": "PA без DPD",
    "ILA DPD": "PA с ILA DPD",
    "direct DPD": "PA с direct-refined DPD",
}
fig, ax = plt.subplots(figsize=(15.5, 6.3))

for name, (frequency, power) in psd_smooth.items():
    smooth_db = 10 * np.log10(np.maximum(power, 1e-30) / reference_peak)
    style = {
        "color": colors[name],
        "linestyle": "--" if name == "ideal" else "-",
        "linewidth": 2.2 if name == "ideal" else 1.8,
    }
    ax.plot(frequency, smooth_db, label=labels[name], **style)

segment_step = PSD_NPERSEG - PSD_NOVERLAP
segment_count = 1 + (len(psd_signals["ideal"]) - PSD_NPERSEG) // segment_step
ax.set(
    xlabel="нормированная частота [циклы/отсчёт]",
    ylabel="PSD [dB относительно пика ideal]",
    title="Нормированная PSD: визуальная диагностика",
)
ax.set_ylim(-100, 5)
ax.legend(loc="best", ncols=2)
ax.text(
    0.01,
    0.02,
    f"Welch: {segment_count} сегм., N={PSD_NPERSEG}; сглаживание={PSD_SMOOTH_BINS}",
    transform=ax.transAxes,
    fontsize=9,
    color="0.35",
)
plt.tight_layout()
plt.show()

```

Интерпретация PSD

Предыскажение приближает спектр выхода к желаемому спектру в рамках frozen evaluator, но локальная близость кривых не заменяет численный спектральный критерий. Без известных полос этот рисунок является визуальной диагностикой спектрального разрастания, а не официальным ACLR/ACPR-измерением.

12. Вычислительная стоимость рабочего DPD

Используется явная конвенция:

- complex multiply = 4 real MUL + 2 real ADD;
- интерполяция одного complex control point = 2 real MUL + 4 real ADD;
- magnitude, comparisons, LUT и state считаются отдельно.

Для ветви выполняются поиск интервала, линейная интерполяция $C_m(r)$ и произведение $x[n - m]C_m(r)$. Число узлов почти не влияет на арифметику интерполяции, но увеличивает coefficient memory и стоимость поиска.

В этом BlackBox-эксперименте каждая ветвь использует собственную задержанную амплитуду $|x[n - m]|$, поэтому модуль вычисляется три раза и стоимость равна 27 MUL. Отдельная конфигурация с общей текущей амплитудой $|x[n]|$ могла бы быть дешевле, но здесь она была отвергнута ablation-экспериментом из-за худшего качества.

Что означают эти числа

Считается только то, что нужно выполнять для каждого нового I/Q-отсчёта. Обучение PA evaluator, ILA и direct refinement в этот подсчёт не входят: они выполняются при калибровке, а не в рабочем потоке. LUT read означает чтение готового коэффициента из небольшой таблицы, а state — хранение двух предыдущих комплексных отсчётов.

```
In [25]: def spline_fast_path_complexity(branches, knot_count):
    """Count operations under the project's explicit scalar convention."""
    branch_count = len(branches)
    unique_envelope_delays = len({envelope_delay for _, envelope_delay in branches})
    maximum_delay = max(max(pair) for pair in branches)
    return {
        "real_multiplications": 6 * branch_count + 3 * unique_envelope_delay,
        "real_additions": 8 * branch_count - 2 + 2 * unique_envelope_delays,
        "magnitude_operations": unique_envelope_delays,
        "comparisons": int(np.ceil(np.log2(knot_count))) * unique_envelope_c,
        "LUT_reads": 2 * branch_count,
        "stored_complex_coefficients": branch_count * knot_count,
        "stored_real_coefficients": 2 * branch_count * knot_count,
        "state_complex_values": maximum_delay,
        "state_real_values": 2 * maximum_delay,
    }

    complexity = spline_fast_path_complexity(branches, knot_count)
    print(json.dumps(complexity, ensure_ascii=False, indent=2))
    print("Параметры других моделей здесь не рисуются рядом с MUL/sample: это ра
```

```
{
  "real_multiplications": 27,
  "real_additions": 28,
  "magnitude_operations": 3,
  "comparisons": 15,
  "LUT_reads": 6,
  "stored_complex_coefficients": 72,
  "stored_real_coefficients": 144,
  "state_complex_values": 2,
  "state_real_values": 4
}
```

Параметры других моделей здесь не рисуются рядом с MUL/sample: это разные величины.

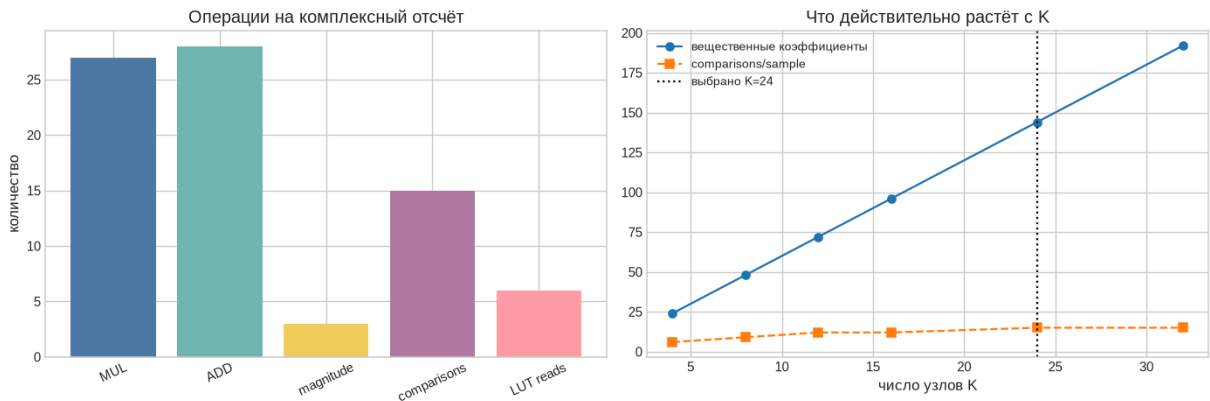
```
In [26]: # Визуальная раскладка операций и памяти выбранной конфигурации.
fig, axes = plt.subplots(1, 2, figsize=(12.8, 4.4))
operation_names = ["MUL", "ADD", "magnitude", "comparisons", "LUT reads"]
operation_values = [
    complexity["real_multiplications"],
    complexity["real_additions"],
    complexity["magnitude_operations"],
    complexity["comparisons"],
    complexity["LUT_reads"],
]
axes[0].bar(operation_names, operation_values,
            color=["#4C78A8", "#72B7B2", "#F2CF5B", "#B279A2", "#FF9DA6"])
axes[0].set(title="Операции на комплексный отсчёт", ylabel="количество")
axes[0].tick_params(axis="x", rotation=25)

k_values = np.array(K_CANDIDATES)
stored_reals = 2 * len(branches) * k_values
comparison_count = np.ceil(np.log2(k_values)).astype(int) * len({d for _, d
axes[1].plot(k_values, stored_reals, "o-", label="вещественные коэффициенты")
```

```

axes[1].plot(k_values, comparison_count, "s--", label="comparisons/sample")
axes[1].axvline(knot_count, color="black", ls=":", label=f"выбрано K={knot_count}")
axes[1].set(xlabel="число узлов K", title="Что действительно растёт с K")
axes[1].legend()
plt.tight_layout()
plt.show()

```



13. Потокковая реализация и воспроизводимость

Проверяются свойства, критичные для реального последовательного выполнения:

1. DPD получает `val_x`, а не `measured val_y`;
2. фазовая эквивариантность выполняется численно;
3. цельная и chunked обработка совпадают при переносе двух отсчётов истории;
4. состояние сбрасывается между независимыми records;
5. validation и predistorted drive не выходят за amplitude support train;
6. DPD действительно уменьшает ошибку frozen evaluator.

```

In [27]: # 1) Данные и модель конечны.
assert np.all(np.isfinite(train_x)) and np.all(np.isfinite(val_x))
assert np.all(np.isfinite(pa_coefficients)) and np.all(np.isfinite(refined_co
assert refined_dpd_coefficients.shape == (len(branches), knot_count)

# 2) Правильное deployment-направление: DPD получает val_x, а не val_y.
replay_from_desired = predict_spline_memory(val_x, knots, branches, refined_
assert np.allclose(replay_from_desired, predistorted_refined_val)

# 3) Фазовая эквивариантность  $D(e^{j\phi}x) = e^{j\phi}D(x)$ .
phi = 0.731
rotated = np.exp(1j * phi) * val_x[:4000]
lhs = predict_spline_memory(rotated, knots, branches, refined_dpd_coefficient
rhs = np.exp(1j * phi) * predict_spline_memory(val_x[:4000], knots, branches
assert np.max(np.abs(lhs - rhs)) < 1e-12

# 4) Потокковая проверка: разбиваем validation на произвольные chunks.
def predict_streaming(signal, chunk_sizes):
    """Run one independent record while carrying only causal history."""
    history_size = max(max(pair) for pair in branches)

```

```

history = np.zeros(history_size, dtype=np.complex128)
output = []
start = 0
for size in chunk_sizes:
    chunk = np.asarray(signal[start:start + size], dtype=np.complex128)
    padded = np.concatenate((history, chunk))
    piece = predict_spline_memory(padded, knots, branches, refined_dpd_coef)
    output.append(piece)
    history = padded[-history_size:] if history_size else history
    start += size
if start < signal.size:
    chunk = np.asarray(signal[start:], dtype=np.complex128)
    padded = np.concatenate((history, chunk))
    output.append(predict_spline_memory(padded, knots, branches, refined_dpd_coef))
return np.concatenate(output)

streamed = predict_streaming(val_x, [1, 8, 64, 257, 1024])
reference_stream = predict_spline_memory(val_x, knots, branches, refined_dpd_coef)
assert np.max(np.abs(streamed - reference_stream)) < 1e-12

# 5) Reset на границе независимого record: history train не попадает в validation
record_a, record_b = val_x[:3000], val_x[3000:7000]
separate_b = predict_streaming(record_b, [17, 131, 509])
repeated_b = predict_streaming(record_b, [1, 29, 777])
direct_b = predict_spline_memory(record_b, knots, branches, refined_dpd_coef)
assert np.max(np.abs(separate_b - direct_b)) < 1e-12
assert np.max(np.abs(repeated_b - direct_b)) < 1e-12
concatenated = predict_spline_memory(np.concatenate((record_a, record_b)), knots, branches, refined_dpd_coef)
# При отсутствии reset первые memory samples второго record обычно отличаются от direct
assert np.max(np.abs(concatenated[record_a.size:record_a.size + DPD_WARMUP] - direct_b[:DPD_WARMUP])) > 0

# 6) Никакого незаявленного extrapolation: validation desired и drive внутри
assert np.max(np.abs(val_x)) <= knots[-1] * (1 + 1e-12)
assert np.max(np.abs(predistorted_refined_val)) <= np.max(np.abs(train_x)) * 1.05

# 7) DPD действительно уменьшает frozen-surrogate error power.
assert refined_mse < dpd_mse < no_dpd_mse
print("ALL SELF-TESTS PASSED")
print(f"phase equivariance max error: {np.max(np.abs(lhs-rhs)):.3e}")
print(f"streaming max error: {np.max(np.abs(streamed-reference_stream)):.3e}")

```

ALL SELF-TESTS PASSED
phase equivariance max error: 3.357e-16
streaming max error: 0.000e+00

```

In [28]: # Машиночитаемый итог validation-части полного запуска.
research_summary = {
    "dataset": "BlackBox capture",
    "train_samples": int(train_x.size),
    "validation_samples": int(val_x.size),
    "pa_evaluator_nmse_db": pooled_nmse_db(pa_val_hat, val_y, PA_WARMUP),
    "no_dpd_nmse_db": surrogate_no_dpd["nmse_db"],
    "ila_spline_nmse_db": dpd_metrics["nmse_db"],
    "direct_refined_nmse_db": refined_metrics["nmse_db"],

```

```
"direct_gain_over_ila_db": direct_gain_over_ila_db,
"direct_gain_over_no_dpd_db": direct_gain_over_no_dpd_db,
"branches": branches,
"knot_count": knot_count,
"ila_ridge": ridge,
"complexity": complexity,
"segment_direct_gain_over_ila_db": segment_gain_ila_to_direct.tolist(),
"streaming_max_error": float(np.max(np.abs(streamed - reference_stream))),
"phase_equivariance_max_error": float(np.max(np.abs(lhs - rhs))),
"direct_model_sha256": DIRECT_MODEL_SHA256,
}
print(json.dumps(research_summary, ensure_ascii=False, indent=2))
```

```

{
  "dataset": "BlackBox capture",
  "train_samples": 92000,
  "validation_samples": 23000,
  "pa_evaluator_nmse_db": -28.590449265757027,
  "no_dpd_nmse_db": -16.010350840472285,
  "ila_spline_nmse_db": -17.192240437598432,
  "direct_refined_nmse_db": -21.587580088478152,
  "direct_gain_over_ila_db": 4.39533965087972,
  "direct_gain_over_no_dpd_db": 5.577229248005867,
  "branches": [
    [
      0,
      0
    ],
    [
      1,
      1
    ],
    [
      2,
      2
    ]
  ],
  "knot_count": 24,
  "ila_ridge": 0.0001,
  "complexity": {
    "real_multiplications": 27,
    "real_additions": 28,
    "magnitude_operations": 3,
    "comparisons": 15,
    "LUT_reads": 6,
    "stored_complex_coefficients": 72,
    "stored_real_coefficients": 144,
    "state_complex_values": 2,
    "state_real_values": 4
  },
  "segment_direct_gain_over_ila_db": [
    4.560822597887139,
    4.257653550904312,
    4.399862316078259,
    4.430407176479701,
    4.423758816890388,
    4.257956529299218,
    4.40702276270385,
    4.407088868196325
  ],
  "streaming_max_error": 0.0,
  "phase_equivariance_max_error": 3.356589068483857e-16,
  "direct_model_sha256": "ddba7f70b63de3c9d3f72991f965a67b7f912795f1cea25561
08e1a31a8eb222"
}

```

14. Untouched waveform in fixed-point

После выбора iterative direct-кандидата до test access были зафиксированы SHA-256 модели и evaluator, warm-up, число сегментов и acceptance gate. В one-shot процедуре открыт только desired x_{test} ; measured y_{test} не открывался. Это проверяет перенос на новую последовательность, но всё ещё через тот же software PA evaluator.

Fixed-point масштабы определены только по train. Проверяются 16/14/12-bit входы и коэффициенты, широкие аккумуляторы, насыщения, столкновения кодов узлов и точное совпадение цельной/кусовой обработки. Это bit-accurate Python reference, а не RTL/HLS timing.

```
In [29]: # Агрегированные evidence-артефакты; test samples в notebook не встроены.
FIXED_JSON_B64 = """ewogICJzdGFnZSI6ICJpdGVyYXRpdmVfZGlyZWNOX2ZpeGVkX3BvaW50
FIXED_JSON_SHA256 = "fc1b4b7f7bd3791daba66c62df458f6842875ac0853eaf9e9e72a99
SEALED_JSON_B64 = """ewogICJzY2h1bWVfdmVyc2lvbiI6IDEsCiAgImFydGlmYWN0X3R5cGU
SEALED_JSON_SHA256 = "6a52bc8605ced498eb656a3937b5047e51d65ee5bac0ef5bf72c66
fixed_report = decode_json_payload(FIXED_JSON_B64, FIXED_JSON_SHA256)
sealed_report = decode_json_payload(SEALED_JSON_B64, SEALED_JSON_SHA256)

validation_scores = [surrogate_no_dpd["nmse_db"], dpd_metrics["nmse_db"],
                      refined_metrics["nmse_db"]]
sealed_scores = [
    sealed_report["metrics"]["no_dpd"]["pooled_complex_nmse_db"],
    sealed_report["metrics"]["ila_spline"]["pooled_complex_nmse_db"],
    sealed_report["metrics"]["iterative_direct_spline"]["pooled_complex_nmse
]

fig, axes = plt.subplots(1, 2, figsize=(14.5, 4.8))
positions = np.arange(3)
width = 0.36
axes[0].bar(positions - width/2, validation_scores, width=width,
             label="validation", color="#9ECAE1")
axes[0].bar(positions + width/2, sealed_scores, width=width,
             label="untouched waveform", color="#3182BD")
axes[0].set_xticks(positions, ["без DPD", "ILA", "direct"])
axes[0].set_ylabel("pooled complex NMSE [dB]",
                  title="Перенос результата на untouched waveform")
axes[0].set_ylim(min(validation_scores + sealed_scores) - 1,
                 max(validation_scores + sealed_scores) + 1)
axes[0].legend()

bit_labels = ["float"] + [str(row["bits"]) for row in fixed_report["formats"]]
bit_nmse = [fixed_report["float_reference"]["cascade_nmse_db"]] + [
    row["cascade_nmse_db"] for row in fixed_report["formats"]]
]
axes[1].plot(bit_labels, bit_nmse, "o-", color="tab:blue")
axes[1].set(xlabel="числовой формат", ylabel="validation NMSE [dB]",
            title="Fixed-point: 12 bit сохраняет качество")
for index, value in enumerate(bit_nmse):
    axes[1].annotate(f"{value:.4f}", (index, value), xytext=(0, 7),
                    textcoords="offset points", ha="center", fontsize=8)

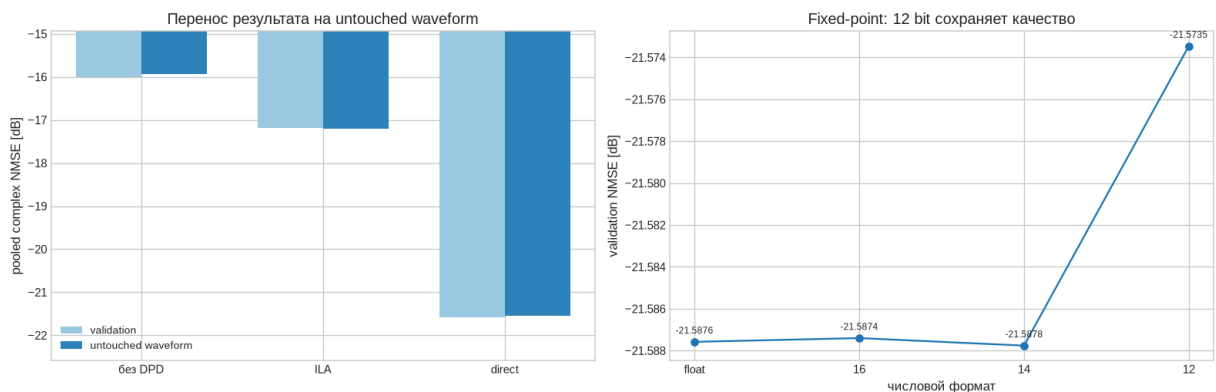
plt.tight_layout()
plt.show()
```

```

assert sealed_report["acceptance_pass"]
assert sealed_report["test_output_opened"] is False
assert min(sealed_report["derived"]["segment_gains_over_ila_db"]) > 0
assert fixed_report["all_streaming_checks_passed"]
assert all(row["coefficient_saturation_count"] == 0 for row in fixed_report["f
assert all(row["knot_code_collision_count"] == 0 for row in fixed_report["f

print(f"Untouched test gain over ILA: {sealed_report['derived']['gain_over_i
print(f"Untouched test gain over no DPD: {sealed_report['derived']['gain_ove
print(f"Minimum test-segment gain over ILA: {min(sealed_report['derived']['s
print("12-bit degradation:",
      next(row["cascade_degradation_vs_float_db"] for row in fixed_report["f
          if row["bits"] == 12), "dB")
print("Test output opened:", sealed_report["test_output_opened"])

```



Untouched test gain over ILA: 4.336798 dB
 Untouched test gain over no DPD: 5.608398 dB
 Minimum test-segment gain over ILA: 4.243725 dB
 12-bit degradation: 0.014125558795104354 dB
 Test output opened: False

15. Выводы

Что показал эксперимент

- Наша трёхветвенная spline-memory DPD после начальной ILA-калибровки дала **-17.192 dB**, а direct refinement тех же коэффициентов этой же модели достиг **-21.588 dB** на validation: выигрыш **4.395 dB** без изменения fast path.
- Первое direct-обновление принесло основную часть выигрыша; три последующие итерации добавили только около 0.113 dB. Это признак насыщения локальной оптимизации, а не основание бесконечно увеличивать число итераций.
- На untouched waveform direct-кандидат получил **-21.545 dB** и сохранил преимущество над ILA во всех восьми последовательных сегментах.
- 12-bit integer reference ухудшил NMSE только примерно на **0.014 dB**; насыщений, knot collisions и streaming-расхождений не обнаружено.
- Дополнительные cross-memory, GMP residual, short FIR и другое размещение узлов дали отрицательный или слишком малый выигрыш относительно

СТОИМОСТИ.

Почему выбран именно этот подход

1. **Spline-memory core** сохраняет фазовую эквивариантность, локальную LUT-интерполяцию и прозрачную fixed-point стоимость.
2. **ILA** даёт быстрое устойчивое начальное решение линейной регрессией.
3. **Direct refinement** улучшает калибровку того же baseline и согласует его коэффициенты с реальной целью каскада, но не добавляет ни одной операции в sample-rate тракт.
4. **Ridge, bounded step и train-advisor split** ограничивают нестабильные направления Jacobian и отделяют расчёт обновления от выбора шага.
5. **AM/AM conditional bands и segment-wise NMSE** не скрывают память за одной усреднённой цифрой или красивой линией.
6. **One-shot test и fixed-point gates** проверяют перенос и числовую реализуемость до физического PA.

Что пока не доказано

- подавление в официально определённых спектральных областях;
- результат после подачи predistorted drive на физический PA;
- независимость от ошибок GMP evaluator;
- RTL/HLS synthesis, latency и throughput;
- стандартная EVM без демодуляционного протокола.

16. Основные источники

1. D. R. Morgan, Z. Ma, J. Kim, M. G. Zierdt, J. Pastalan, "A Generalized Memory Polynomial Model for Digital Predistortion of RF Power Amplifiers," *IEEE Transactions on Signal Processing*, 2006. [DOI](#)
2. C. Eun, E. J. Powers, "A New Volterra Predistorter Based on the Indirect Learning Architecture," *IEEE Transactions on Signal Processing*, 1997. [DOI](#)
3. P. P. Campo, A. Brihuega, L. Anttila, M. Turunen, D. Korpi, M. Allén, M. Valkama, "Gradient-Adaptive Spline-Interpolated LUT Methods for Low-Complexity Digital Predistortion," *IEEE Transactions on Circuits and Systems I*, 2020. [arXiv](#) · [DOI](#)
4. Y. Wu, G. D. Singh, M. Beikmirza, L. C. N. de Vreede, M. Alavi, C. Gao, "OpenDPD: An Open-Source End-to-End Learning & Benchmarking Framework for Wideband Power Amplifier Modeling and Digital Pre-Distortion," *ISCAS 2024*. [arXiv](#) · [официальный репозиторий](#)

5. J. Wood, *Behavioral Modeling and Linearization of RF Power Amplifiers*, Artech House, 2014.

Воспроизведение direct refinement

Из корня проекта:

```
python -m experiments.evaluate_blackbox_dpd_hypotheses --stage  
direct
```

```
python -m experiments.evaluate_blackbox_dpd_hypotheses --stage  
direct-robustness
```

```
python -m experiments.evaluate_blackbox_dpd_hypotheses --stage  
direct-iterative
```

```
python -m experiments.evaluate_blackbox_dpd_hypotheses --stage  
fixed-point
```

One-shot test повторно не запускается: его агрегированный отчёт уже связан с предварительной регистрацией и показан выше.